



SHARIF COLLEGE OF ENGINEERING & TECHNOLOGY

Department of Computer Science

FINAL YEAR PROJECT REPORT

AI Powered HR Screening and Document Retrieval System Using RAG

**Submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science (BSCS)**

Submitted By

Sr No.	Student Name	Registration No.	Signature
1	AMNA IKRAM	UET-SHCET-LHR-CS-02	
2	ZAIN UL HASSAN	UET-SHCET-LHR-CS-12	
3	EZZA ANSAR	UET-SHCET-LHR-CS-13	

Supervisor

Ms. Hirra Mustafa

Co-Supervisor

Dr. Mazhar Iqbal

I. Industrial Supervisor Approval Certificate



To Whom It May Concern,

This letter is to recommend the Final Year Project titled **“Intelligent Document Retrieval and Question Answering System using On-Premises AI and Retrieval-Augmented Generation”** undertaken by the following students:

- Zain Ul Hassan 2022-UET-SHCET-LHR-CS-12 37301-9525716-1
- Ezza Ansar 2022-UET-SHCET-LHR-CS-13 35201-7121055-6
- Amna Ikram 2022-UET-SHCET-LHR-CS-02 35202-4463525-8

The project addresses a significant industrial need for secure and efficient document processing through AI and Retrieval-Augmented Generation techniques. It aligns well with current trends in enterprise automation, emphasizing data privacy and intelligent information retrieval.

I endorse this project as their **Industrial Supervisor** and believe it holds strong potential for real-world application in sectors requiring secure document management and analysis.

Supervisor Name: Anjitha Sakthidharan

Designation: CEO

Organization: VenturesSky Ltd

Email: anjitha@venturessky.com

Phone: +447794568221

Sincerely,

Anjitha Sakthidharan

Anjitha Sakthidharan

VenturesSky Ltd.

Venturessky Limited
D&D Accountancy Suite 006, 44-60 Richardshaw Lane, Pudsey, LS28 7UR

II. Declaration

We hereby declare that this Final Year Project Report entitled:

"AI Powered HR Screening and Document Retrieval System Using RAG"

submitted to the **Department of Computer Science, Sharif College of Engineering & Technology**, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Science in Computer Science (BSCS)**, is our original work.

This project has been carried out by us under the supervision of Ms. Hirra Mustafa.

We further declare that:

- The work contained in this report is entirely our own except where appropriate acknowledgment and references have been made.
- All sources of information, including published literature, books, journals, websites, and other materials, have been properly cited in accordance with APA 7th Edition referencing guidelines.
- We accept full responsibility for the authenticity and originality of the work presented in this report.
- Any similarity with previously published work beyond acceptable academic limits is unintentional and subject to the academic policies of Sharif College of Engineering & Technology.

We understand that if any part of this declaration is found to be false or misleading, the College reserves the right to cancel the submission and take disciplinary action in accordance with its academic regulations.

SUPERVISOR'S VERIFICATION

I certify that the above declaration has been read and that, to the best of my knowledge, the work presented in this Final Year Project Report has been carried out by the above-mentioned students under my supervision.

Supervisor Name: Ms. Hirra Mustafa

Designation: _____

Department: Computer Science

Signature: _____

Date: _____

III. Plagiarism Declaration/ Plagiarism Report

We hereby declare that this Final Year Project Report is our original work and has been prepared in accordance with the academic integrity policies of the **Department of Computer Science, Sharif College of Engineering & Technology (SCET)**.

We confirm that all sources of information have been properly acknowledged and cited. Any text, figures, code, or other material taken from published or unpublished sources has been appropriately referenced.

We understand and agree to the following submission requirements:

- **Similarity Score:** The overall similarity index must be **15% or less**.
- **AI Content Score:** The AI-generated content score must be **less than 20%**.

We acknowledge that failure to comply with these requirements or submission of plagiarized work may result in rejection of the project and disciplinary action in accordance with the academic regulations of SCET.

IV. Dedication

To our beloved parents, whose prayers, sacrifices, and endless encouragement made this journey possible, and to our teachers, whose guidance and patience shaped the way we think and work. This project is dedicated to you, with gratitude and love.

V. Acknowledgements

First and foremost, we thank Almighty Allah for granting us the strength, patience, and determination to complete this project.

We are deeply grateful to our supervisor, Ms. Hirra Mustafa, for her constant guidance, valuable feedback, and encouragement at every stage of this work. Her insight kept the project on track and pushed us to do better. We are equally thankful to our co-supervisor, Dr. Mazhar Iqbal, whose technical advice and thoughtful suggestions strengthened both our approach and our understanding.

We also extend our sincere appreciation to our industrial supervisor, Anjitha Sakthidharan (CEO, VenturesSky Ltd), for endorsing the project and offering an industry perspective that helped us ground our work in real-world needs.

Our gratitude goes to the Department of Computer Science at Sharif College of Engineering and Technology for providing the resources, facilities, and learning environment that made this work possible, and to the faculty members whose feedback during evaluations helped us refine the system.

Finally, we thank our families and friends for their unconditional love, patience, and support throughout this journey. Their belief in us was a constant source of motivation.

Abstract

Human resource teams that manage large collections of unstructured documents spend considerable time manually reviewing and screening resumes, with inconsistent results, while keyword search fails to surface semantically relevant content. This project presents Hireflow, an AI-powered HR screening and document retrieval system built on Retrieval-Augmented Generation. Documents in PDF, Word, and scanned-image form are processed through an OCR and text-extraction pipeline, then chunked, embedded with sentence-transformer models, and indexed in a vector database. A hybrid retrieval engine fuses vector, lexical, metadata, and fuzzy signals, and a retrieval-augmented generation service returns concise, source-attributed answers while declining questions that fall outside the document collection. Candidate resumes are ranked against job criteria using a transparent weighted scoring model, and intake is automated through Gmail synchronization. The system is implemented as a FastAPI backend with PostgreSQL, Redis, Celery, and MinIO, paired with a React frontend. The result is a single self-hostable platform delivering semantic search, grounded question answering, and explainable candidate ranking.

Keywords: *Retrieval-Augmented Generation, Semantic Search, Resume Screening, Optical Character Recognition, Vector Database, Human Resources.*

Table of Contents

I. Industrial Supervisor Approval Certificate.....	ii
II. Declaration.....	iii
III. Plagiarism Declaration/ Plagiarism Report.....	iv
IV. Dedication.....	v
V. Acknowledgements.....	vi
Abstract.....	vii
List of Abbreviations.....	xiv
CHAPTER 1: INTRODUCTION.....	1
1.1 Background.....	1
1.2 Problem Statement.....	2
1.3 Objectives.....	2
1.4 Scope of Project.....	3
1.5 Significance of Project.....	3
1.6 Report Organization.....	4
CHAPTER 2: LITERATURE REVIEW.....	5
2.1 Overview of Domain.....	5
2.2 Existing Systems / Related Work.....	5
2.3 Comparison of Existing Systems.....	7
2.4 Research Gap.....	7
2.5 Proposed Solution.....	8
CHAPTER 3: SYSTEM ANALYSIS & REQUIREMENTS.....	10
3.1 Feasibility Study.....	10
3.1.1 Technical Feasibility.....	10
3.1.2 Economic Feasibility.....	10
3.1.3 Operational Feasibility.....	11
3.1.4 Schedule Feasibility.....	11
3.2 Functional Requirements.....	11
3.3 Non-Functional Requirements.....	14
3.3.1 Performance.....	14
3.3.2 Security and Privacy.....	14
3.3.3 Scalability.....	15

3.3.4 Usability.....	15
3.3.5 Reliability.....	15
3.3.6 Maintainability.....	15
3.4 Tools and Technologies Used.....	15
3.5 Use Cases / User Stories.....	18
3.6 Use Cases Table.....	20
3.7 Development Methodology.....	22
CHAPTER 4: SYSTEM DESIGN.....	24
4.1 System Architecture Description & Design.....	24
4.2 Data Flow Diagrams.....	26
4.3 UML Diagrams.....	28
4.3.1 Class Diagram.....	28
4.3.2 Activity Diagram.....	29
4.3.3 Component Diagram.....	30
4.3.4 Sequence Diagram.....	30
4.3.5 State Machine Diagram.....	31
4.4 Database Design (ERD, Schema, Data Dictionary).....	31
4.5 UI/UX Design.....	36
CHAPTER 5: IMPLEMENTATION.....	42
5.1 Development Environment Setup.....	42
5.2 Implementation Details.....	43
5.2.1 Layered Backend.....	43
5.2.2 Request Validation and Error Handling.....	44
5.2.3 Ingestion Pipeline.....	52
5.2.4 Search and RAG.....	53
5.2.5 Conversations and Follow-up Questions.....	54
5.2.6 Authentication and Storage.....	54
5.2.7 Gmail Synchronisation.....	55
5.2.8 Frontend and API Client.....	55
5.3 Code Snippets with Explanation.....	56
5.3.1 Timing-Safe Authentication.....	56
5.3.2 Weighted Candidate Scoring.....	56
5.3.3 Self-Documenting Routes.....	57

5.4 Database Implementation.....	58
CHAPTER 6: TESTING & RESULTS.....	59
6.1 Testing Strategy.....	59
6.2 Test Cases and Results.....	59
6.3 Performance Evaluation.....	61
6.3.1 Retrieval Latency.....	62
6.3.2 Ingestion Throughput.....	62
6.3.3 Embedding and LLM Response Times.....	63
6.4 Results and Discussion.....	63
CHAPTER 7: CONCLUSION & FUTURE WORK.....	65
7.1 Conclusion.....	65
7.2 Limitations.....	65
7.3 Future Enhancements.....	66
REFERENCES.....	67
APPENDICES.....	69
Appendix A: User Manual.....	69
Appendix B: Sample Code.....	69
Appendix C: Gantt Chart (FYP-I and FYP-II).....	69

List of Figures

Figure 3.1: Use Case Diagram.....	18
Figure 3.2: Development Methodology Flowchart.....	22
Figure 4.1: System Architecture.....	26
Figure 4.2: DFD Level 0 (Context Diagram).....	27
Figure 4.3: DFD Level 1.....	27
Figure 4.4: DFD Level 2.....	28
Figure 4.5: Class Diagram.....	28
Figure 4.6: Activity Diagram.....	29
Figure 4.7: Component Diagram.....	30
Figure 4.8: Sequence Diagram (Streaming RAG Query).....	30
Figure 4.9: Document Status State Machine.....	31
Figure 4.10: Entity-Relationship Diagram.....	31
Figure 4.11: Dashboard overview.....	37
Figure 4.12: Documents management with upload and processing status.....	37
Figure 4.13: Semantic search across the document library with match scores and skill filters.....	38
Figure 4.14: Ask Hireflow, conversational question answering grounded in the library with source citations.....	38
Figure 4.15: Candidate screening list with match scores.....	39
Figure 4.16: Job creation with required skills, preferred skills, and an experience range.....	39
Figure 4.17: Document detail view with the in-app viewer and the extracted metadata	40
Figure 4.18: Activity log showing the audit trail of actions.....	40
Figure 4.19: Settings, with the theme, password change, and the Gmail connection. .	41
Figure 5.1: Invalid credentials (HTTP 401).....	46
Figure 5.2: Invalid token (HTTP 401).....	47
Figure 5.3: Account disabled (HTTP 403).....	47
Figure 5.4: Forbidden (HTTP 403).....	47
Figure 5.5: Not found (HTTP 404).....	48
Figure 5.6: Document not indexed (HTTP 404).....	48
Figure 5.7: Email already registered (HTTP 409).....	48
Figure 5.8: Invalid status transition (HTTP 409).....	49

Figure 5.9: Resume already attached (HTTP 409).....	49
Figure 5.10: File too large (HTTP 413).....	49
Figure 5.11: Unsupported file type (HTTP 415).....	50
Figure 5.12: LLM rate limited (HTTP 429).....	50
Figure 5.13: Gmail authorisation error, served as a redirect to the settings page.....	50
Figure 5.14: Service unavailable (HTTP 503).....	51
Figure 5.15: LLM timeout (HTTP 504).....	51
Figure 5.16: Validation error (HTTP 422).....	52

List of Tables

Table 2.1: Review of Existing Systems and Related Work.....	6
Table 2.2: Comparison of Representative Systems.....	7
Table 2.3: Research Gap Summary.....	8
Table 3.1: Functional Requirements (FR01-FR22).....	11
Table 3.2: Tools and Technologies.....	16
Table 3.3: Summary of Use Cases (UC-01-UC-12).....	19
Table 3.4: Sample Use Case, User Login.....	20
Table 4.1: users Data Dictionary.....	32
Table 4.2: documents Data Dictionary.....	32
Table 4.3: jobs Data Dictionary.....	33
Table 4.4: candidates Data Dictionary.....	34
Table 5.1: Development Environment Services and Ports.....	43
Table 5.2: Domain Error to HTTP Status Mapping.....	44
Table 5.3: Document Ingestion Pipeline Stages.....	52
Table 6.1: Test Cases and Results.....	61
Table 6.2: Test Case Traceability Matrix.....	62
Table 6.3: Measured Performance (Development Stack).....	63

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
ATS	Applicant Tracking System
CCPA	California Consumer Privacy Act
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram
FR	Functional Requirement
FTS	Full-Text Search
GDPR	General Data Protection Regulation
HR	Human Resources
JWT	JSON Web Token
LLM	Large Language Model
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
OAuth	Open Authorization
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
RRF	Reciprocal Rank Fusion
SPA	Single-Page Application
SRS	Software Requirements Specification
SSE	Server-Sent Events
UC	Use Case
UI/UX	User Interface / User Experience
UML	Unified Modeling Language

CHAPTER 1: INTRODUCTION

Recruitment and document management are among the most document-intensive functions in any organisation. Yet they remain heavily manual, slow, and inconsistent. This chapter introduces Hireflow, an AI-Powered HR Screening and Document Retrieval System that applies Retrieval-Augmented Generation (RAG) to unstructured document collections, letting HR personnel search, question, and screen documents in natural language. The sections that follow set out the background and motivation, state the problem being solved, define the objectives and scope, weigh the significance of the work, and outline how the rest of the report is organised.

1.1 Background

Modern organisations accumulate large volumes of unstructured documents: resumes, reports, contracts, and scanned correspondence, most of it arriving as PDF files, Microsoft Word documents, or scanned images. Extracting value from this material has traditionally depended on manual reading and keyword-based search. Both scale poorly as collections grow. In human resources the pressure is acute. Recruiters commonly review dozens of resumes per open position, and industry practice reports that HR teams spend the majority of their working time on manual resume review, averaging around twenty-three hours of effort to fill a single position. Keyword search only compounds the problem, since it matches literal tokens rather than meaning; a query for “machine learning engineer” fails to surface a strong candidate who wrote “ML researcher” or “deep learning specialist.”

Two families of technology have now matured enough to address these limitations. The first is document understanding and optical character recognition (OCR). Managed cloud services such as Google Cloud Document AI [1], Amazon Textract [2], and Azure Form Recognizer [3], along with open-source engines such as Tesseract [4] and libraries such as PyMuPDF [5], make it possible to extract structured text from PDFs, Word files, and scanned images. The second is the pairing of dense vector embeddings with large language models (LLMs). Vector similarity search libraries such as FAISS [6], together with transformer-based document and language models [7][8], enable semantic retrieval, where documents are matched by meaning rather than exact wording.

Retrieval-Augmented Generation (RAG) [9] ties these two strands together. It is a technique for using a language model, not a product or a model in its own right, and the idea is the difference between a closed-book and an open-book exam. Asked a question on its own, a language model answers from memory: it writes fluently, but it has never seen a private document collection, and when it does not know something it tends to produce a plausible-sounding invention. RAG makes the exam open-book. The system first retrieves the passages from the collection that are most relevant to the

question, then gives the model those passages along with the question and instructs it to answer from that material only. The answer is still written in ordinary language, but the facts come from the retrieved passages, so every statement can be traced back to the document it came from. Hireflow builds on this foundation, using sentence-transformers [10] for embeddings, ChromaDB as the vector store, and Anthropic Claude [11] as the answer-generating LLM.

1.2 Problem Statement

HR personnel who manage large, unstructured document collections face several compounding difficulties. Manual review of resumes and other documents is slow and absorbs the bulk of recruiter effort, and because different reviewers apply inconsistent criteria, it introduces bias and compliance risk. Keyword-based search and rigid, rule-based Applicant Tracking System (ATS) filters miss semantically relevant content, along with qualified candidates whose phrasing does not match the exact query terms. The information needed for a decision is scattered across email inboxes, shared folders, and disconnected systems, so retrieving a specific fact from a large corpus is slow. Existing tools rarely let a user ask a plain-language question over their own documents and receive a concise, source-attributed answer. What is needed is a single system that ingests heterogeneous documents, understands them semantically, answers natural-language questions with provenance, and automatically screens and ranks candidates against defined job criteria.

1.3 Objectives

The project pursues the following specific and measurable objectives:

1. **Ingest and process heterogeneous documents:** accept PDF, Microsoft Word, and scanned image files and extract their text using PyMuPDF [5] and Tesseract/pytesseract OCR [4], targeting text-extraction accuracy above 95% on standard printed documents.
2. **Enable semantic search and retrieval:** index all documents as dense vector embeddings (sentence-transformers) in a vector database (ChromaDB) so that natural-language queries return relevance-ranked results, with typical search responses returned in under two seconds.
3. **Provide grounded question answering via RAG:** answer plain-language questions over the document collection using Retrieval-Augmented Generation [9] with an LLM (Anthropic Claude), returning cited, source-attributed answers with a target answer-relevance of at least 85% and end-to-end response times under ten seconds.
4. **Automate resume screening and ranking:** match and rank candidate resumes against user-defined job criteria (skills, education, experience), reducing manual screening effort by roughly 70-80% and processing on the order of 100 resumes per hour.

5. **Automate candidate intake and reporting:** sync resume attachments from Gmail using OAuth 2.0 [12], and support shortlisting, rejection, activity logging, and export of candidate data to Excel for compliance and offline analysis.

1.4 Scope of Project

Hireflow is a standalone, web-based system for HR personnel. Within scope for the first version are document upload and batch ingestion; OCR and text extraction for PDF, Word, and scanned image files; automatic document classification and metadata generation; semantic search with filtering by document type, date, skills, and role; RAG-based question answering with source citations; job posting creation and management; automated candidate screening and ranking against job criteria; Gmail integration for resume collection via OAuth 2.0; export of shortlisted candidates to Excel; and activity logging with audit trails. The system is delivered as a FastAPI (Python 3.12) backend with Celery and Redis for background processing, PostgreSQL 15 for relational data, ChromaDB for vector storage, and MinIO for object storage, paired with a React 19 and TypeScript frontend built with Vite, Tailwind, and shadcn/ui.

The system supports two roles: HR personnel (the primary user class) and an administrator role for user management, both governed by role-based access control. An external Email Service acts as a supporting actor for automated resume synchronisation. Explicitly out of scope for version 1.0 are third-party ATS integrations, processing of video or audio files, native mobile applications for iOS and Android, and multi-language support beyond English.

1.5 Significance of Project

Hireflow's significance lies in consolidating capabilities that were previously available only through fragmented, expensive, or cloud-locked services into a single coherent platform tailored to HR workflows. Replacing keyword matching with semantic retrieval and RAG lets the system surface qualified candidates and relevant documents that literal search would miss. This improves the quality and consistency of shortlists while reducing bias from divergent reviewer criteria. By automating resume intake from Gmail and ranking candidates against explicit job criteria, the system targets a substantial reduction in manual screening effort, freeing recruiters for interviews and judgment-heavy work. Grounded, source-attributed answers address a key weakness of naive LLM use, namely unverifiable output, by tying every answer back to the documents it came from; this matters for trust, auditability, and compliance. Academically, the project shows how OCR, dense-vector retrieval, and LLM-based generation can be integrated into a production-shaped, service-oriented architecture.

1.6 Report Organization

This report is organised into seven chapters. Following this introduction, Chapter 2 reviews the literature on document understanding, semantic search, and AI-assisted recruitment, compares the existing systems, and identifies the research gap. Chapter 3 covers the analysis and requirements, from the feasibility study through the functional and non-functional requirements, the tools and technologies adopted, and the use cases. Chapter 4 presents the system design, including the layered architecture, the data-flow diagrams, the UML models, the database design, and the user interface. Chapter 5 describes the implementation, covering the development environment, the backend and frontend, representative code, and the database. Chapter 6 reports the testing strategy, the test cases and their results, and the performance evaluation. Chapter 7 concludes with the outcomes of the project, its limitations, and directions for future work.

CHAPTER 2: LITERATURE REVIEW

This chapter reviews the technologies and prior systems relevant to AI-powered document retrieval and HR screening. It first frames the problem domain, then examines existing systems and related work: traditional Applicant Tracking Systems, keyword search, cloud document-understanding services, RAG frameworks, transformer-based document understanding, vector similarity search, and OCR. From there it compares the approaches, identifies the research gap that motivates the project, and presents Hireflow as the proposed solution.

2.1 Overview of Domain

This project draws on three fields: document understanding, information retrieval, and applied natural language processing for recruitment. Document understanding converts unstructured inputs such as PDFs, Word files, and scanned images into machine-readable text and metadata, typically using OCR and layout analysis [1][2][3][5][4]. Information retrieval has evolved from lexical, keyword-based matching toward dense semantic retrieval, in which text is encoded as high-dimensional embedding vectors and compared by similarity, so that conceptually related content surfaces even without shared keywords [8][6]. Retrieval-Augmented Generation combines retrieval with generative language models, grounding an LLM's response in retrieved passages to produce accurate, citable answers rather than unsupported text [9]. Recruitment applies these ideas directly: Applicant Tracking Systems and AI recruitment platforms parse resumes, match candidates to job requirements, and manage hiring pipelines. Hireflow draws on all three fields to serve HR personnel working over unstructured document collections.

2.2 Existing Systems / Related Work

A range of commercial products, cloud services, and open-source tools address parts of the problem that Hireflow tackles. Traditional ATS platforms such as Greenhouse [13] and Workday [14] automate applicant pipelines but lean heavily on keyword filters and structured fields; AI recruitment platforms such as Eightfold AI [15] apply deep learning to candidate matching. Cloud document-understanding services (Google Cloud Document AI [1], Amazon Textract [2], Azure Form Recognizer [3]) extract text and structure at scale, though they are proprietary and usage-priced. RAG frameworks such as LangChain [16] coordinate retrieval-augmented pipelines, while transformer-based document and language models [7][8] support both understanding and generation. Lower down the stack, vector search libraries such as FAISS [6] and OCR engines such as Tesseract [4] supply the retrieval and text-extraction primitives.

The representative systems reviewed for this project are summarised in Table 2.1, each characterised by its research problem, method and tools, key features, limitations, and relevance to Hireflow.

Table 2.1: Review of Existing Systems and Related Work

#	System	Research Problem	Methodology & Tools	Key Features	Limitations	Relevance to Hireflow
1	Smith / Tesseract community [4]	Reading text from scanned images and PDFs	LSTM-based OCR over rasterised document images; Tesseract and pytesseract	Free, offline, multi-format OCR	Accuracy drops on poor scans and handwriting; no semantic understanding	Provides the OCR stage of Hireflow's ingestion pipeline
2	Greenhouse Software	Managing and filtering high volumes of job applicants	Structured fields with keyword/boolean rules over parsed resumes; commercial ATS and resume parser	Pipeline management, keyword filters, structured candidate records	Keyword matching misses semantics; rigid rules; limited free-form question answering	Establishes the ATS baseline that Hireflow improves on with semantic ranking
3	Eightfold AI	Bias and inefficiency in candidate-job matching	Learned embeddings of candidates and roles for similarity matching; proprietary deep-learning models	AI candidate matching, skills inference, ranking	Proprietary, closed, and costly, with limited transparency and provenance	Validates ML-based ranking and motivates open, explainable ranking with citations
4	Johnson, Douze & Jegou / FAISS [6]	Fast similarity search over very large collections of vectors	Approximate nearest-neighbour indexing of dense embeddings; the FAISS library	Efficient, scalable approximate nearest-neighbour search	A library only, no ingestion, OCR, or application layer	Supports semantic retrieval; Hireflow uses ChromaDB in the same role
5	Google Cloud Document AI, Amazon Textract, Azure Form Recognizer [1][2][3]	Extracting structured data from unstructured documents	Deep OCR with layout analysis and form/entity extraction, delivered as managed cloud APIs	High-accuracy OCR, entity and table extraction, scalability	Proprietary, per-use cost, cloud lock-in, data leaving the premises	Motivates an on-premise, open OCR pipeline (PyMuPDF + Tesseract)
6	Hugging Face / Transformer community [7][8]	Understanding document text and semantics	Pretrained transformer encoders for embeddings and LLMs for generation; Transformers and sentence-transformers	Semantic embeddings, classification, generation	Compute-heavy, and general models need domain adaptation	The basis for Hireflow's embeddings and LLM answer generation
7	Lewis et al. / LangChain [9]	LLMs hallucinate and lack access to private or	Retrieve relevant chunks from a vector store, then condition an LLM on them;	Grounded, citable answers over private	Answer quality depends heavily on retrieval and	The core paradigm adopted by Hireflow for document

#	System	Research Problem	Methodology & Tools	Key Features	Limitations	Relevance to Hireflow
		current data	LangChain, vector databases, LLMs	corpora	chunking, and needs tuning	question answering

2.3 Comparison of Existing Systems

Each reviewed system solves one facet of the problem well, but none offers an integrated, HR-focused, provenance-aware solution. Traditional ATS platforms and keyword search excel at structured pipeline management and exact-match filtering, yet their lexical matching misses semantically equivalent phrasing, and their rigid rules cannot answer free-form questions. AI recruitment platforms such as Eightfold AI show the value of learned matching, but they are proprietary, costly, and opaque, offering little insight into why a candidate was ranked. Cloud document-AI services deliver excellent OCR and extraction accuracy at the price of per-use cost, cloud lock-in, and off-premise data handling that is often unacceptable for sensitive HR records. RAG frameworks and transformer models supply the generative and semantic capabilities, but they are building blocks rather than finished applications and demand substantial integration work. Vector search libraries and OCR engines are similarly low-level. The comparison below summarises these trade-offs.

Table 2.2: Comparison of Representative Systems

System / Approach	Semantic Retrieval	Natural-Language Q&A	Automated Resume Ranking	Provenance / Citations	Open / On-Premise
Traditional ATS (Greenhouse, Workday)	No (keyword)	No	Rule-based only	Limited	Mostly cloud/commercial
AI recruitment (Eightfold AI)	Yes	Limited	Yes (ML)	Weak	No (proprietary)
Cloud Document AI [1][2][3]	Partial	No	No	N/A	No (cloud)
RAG frameworks (LangChain) [9]	Yes	Yes	No (not HR-specific)	Yes (if built)	Yes (self-host)
Hireflow (proposed)	Yes	Yes	Yes (LLM-based)	Yes	Yes

2.4 Research Gap

The comparison shows that no single existing system combines semantic retrieval, grounded natural-language question answering, LLM-based candidate ranking,

automated email-based resume intake, and transparent source attribution in one HR-oriented, self-hostable platform. Each gap below maps directly to a design choice in Hireflow.

Table 2.3: Research Gap Summary

Approach / System	Gap Identified	Proposed Improvement
Keyword search / traditional ATS filters	Literal token matching misses semantically relevant documents and candidates	Semantic RAG retrieval using dense sentence-transformer embeddings in ChromaDB
Rigid rule-based ATS screening	Fixed rules cannot weigh nuanced skills, phrasing, or context; inconsistent criteria	LLM-based candidate ranking against structured job criteria for consistent, context-aware scoring
Document collections without Q&A	No way to ask plain-language questions and get a synthesized answer over one's own documents	RAG chat interface that answers natural-language questions over the indexed corpus
Manual resume collection from inboxes	Resumes scattered across email require manual download and filing	Automated Gmail sync via OAuth 2.0 that ingests resume attachments in the background
Naive LLM answers / opaque AI ranking	Answers and rankings lack verifiable sources, harming trust and auditability	Cited, source-attributed answers and logged, auditable screening decisions

2.5 Proposed Solution

Hireflow is proposed as an integrated, web-based platform that closes these gaps by combining best-of-breed open components into a single HR-focused system. Documents in PDF, Word, and scanned-image form are ingested through an OCR and text-extraction stage built on PyMuPDF [5] and Tesseract/pytesseract [4]. This avoids the cost and cloud lock-in of managed document-AI services [1][2][3] while retaining acceptable accuracy on standard printed material. Extracted text is chunked, embedded with sentence-transformer models [7][8][10], and stored in ChromaDB, which plays the same approximate-nearest-neighbour retrieval role popularized by FAISS [6]. This gives semantic search that goes beyond keyword matching. On top of retrieval, Hireflow implements Retrieval-Augmented Generation [9] with Anthropic Claude as the generation model: for each natural-language question, relevant passages

are retrieved and supplied as grounded context, and the model returns a concise answer with citations back to the source documents. This directly addresses the provenance weakness of naive LLM use and of opaque AI recruitment platforms.

For recruitment, HR personnel define job postings with explicit skills, education, and experience criteria. The system then ranks candidate resumes against those criteria using LLM-based scoring rather than rigid rules, producing consistent and explainable shortlists. Resume intake is automated through Gmail integration over OAuth 2.0, which syncs attachments in the background via Celery and Redis and removes the manual collection step. The platform is realised as a FastAPI (Python 3.12) backend with PostgreSQL 15 for relational data and MinIO for object storage, paired with a React 19 and TypeScript frontend driven by an OpenAPI-generated client; role-based access control, activity logging, and audit trails support compliance in line with recommended requirements-engineering practice [17]. These choices deliver the semantic retrieval, grounded Q&A, explainable ranking, automated intake, and source attribution that no single reviewed system provides.

CHAPTER 3: SYSTEM ANALYSIS & REQUIREMENTS

This chapter presents the analysis behind the design of Hireflow, the AI-Powered HR Screening and Document Retrieval System built on Retrieval-Augmented Generation (RAG). A feasibility study establishes that the system is viable. The chapter then consolidates the functional and non-functional requirements gathered during the Software Requirements Specification (SRS) phase, documents the tools and technologies adopted for implementation, models system behaviour through use cases and user stories, and closes with the development methodology the project followed. These sections turn a familiar problem, slow and manual document review and resume screening, into a concrete and verifiable set of requirements that guides the design and implementation that follow.

3.1 Feasibility Study

A feasibility study was carried out to confirm that Hireflow could be delivered within the constraints of a Final Year Project while still meeting the needs of HR personnel who handle large volumes of unstructured documents. It considers four dimensions: technical, economic, operational, and schedule.

3.1.1 Technical Feasibility

The system is technically feasible because every capability rests on mature, well-documented, open-source components. PyMuPDF, the `unstructured` library, and Tesseract handle text extraction and OCR. ChromaDB serves semantic retrieval over sentence-transformer embeddings. Question answering runs through an LLM provider (Anthropic Claude, with an Ollama fallback) behind a pluggable adapter. The backend is a layered FastAPI application (domain, models, repositories, adapters, services, API) with asynchronous background processing via Celery and Redis; PostgreSQL 15 stores metadata and MinIO stores the original files. AI providers for vision, LLM, and embeddings are selected at runtime through Protocol-based adapters, so the system runs on CPU-only hardware, exploits GPU acceleration where available, and avoids vendor lock-in. A working, integrated prototype spanning 31 API endpoints and a React 19 frontend confirms all of this in practice.

3.1.2 Economic Feasibility

The project is economically feasible. Its technology stack is almost entirely free and open-source software, so the database, vector store, object storage, OCR engine, and web frameworks carry no licensing costs. The one variable operating cost is optional LLM API usage for RAG answering and classification fallback. This stays bounded: the LLM is called only for question answering and low-confidence classification, and an on-premise Ollama provider can replace the hosted model outright. Deployment needs no more than commodity server hardware (16 GB RAM minimum, 32 GB

recommended) or a modest cloud instance. Weighed against the recurring staff hours spent on manual resume screening, the return on investment is clear.

3.1.3 Operational Feasibility

The system fits naturally into existing HR recruitment workflows. Its primary users, HR personnel of moderate technical skill, reach it through a browser-based application that needs no client-side installation and adapts to desktop and tablet screens. Routine tasks such as uploading documents, searching in natural language, creating job postings, screening candidates, and syncing resumes from Gmail mirror what HR staff already do by hand, which keeps the learning curve short. Synchronisation and processing run in the background on their own, so the system removes effort instead of adding operational burden.

3.1.4 Schedule Feasibility

The project is schedule-feasible. A formal SRS bounded the work up front, covering twenty-two functional requirements (FR01-FR22) and twelve use cases (UC-01-UC-12), which fixed scope early. Delivery followed an incremental, feature-by-feature model, one tracked feature per iteration, moving from authentication through document processing, search, RAG, job management, screening, and Gmail sync. This let the system be built and validated in stages within the academic timeline. Optional and future-facing items such as ERP integration were deferred on purpose, protecting the core schedule.

3.2 Functional Requirements

The functional requirements were drawn from the SRS System Features section and grouped by feature area. Each carries a stable identifier (FR01-FR22) and maps to one or more use cases (UC-01-UC-12), giving traceability from requirement to behaviour.

Table 3.1: Functional Requirements (FR01-FR22)

FR-ID	Feature Area	Requirement	Use Case
FR01	Authentication & Access Control	Allow users to log in using email and password	UC-01
FR02	Authentication & Access Control	Allow users to reset their password using a verification mechanism	UC-02
FR03	Authentication & Access Control	Restrict access to features unless the	UC-01

FR-ID	Feature Area	Requirement	Use Case
		user is authenticated	
FR04	Document Upload & Processing	Allow HR personnel to upload files such as resumes and HR documents	UC-03
FR05	Document Upload & Processing	Extract text from uploaded documents using OCR when needed	UC-03
FR06	Document Upload & Processing	Store uploaded documents together with their extracted text for retrieval	UC-03
FR07	Search & Retrieval	Allow users to search documents using keywords or natural language	UC-04
FR08	Search & Retrieval	Allow users to search based on criteria such as skills, job role, and date	UC-05
FR09	Search & Retrieval	Display search results ranked by relevance	UC-04
FR10	Filter Documents	Allow users to filter documents based on criteria (skills, role, date)	UC-05
FR11	Job Creation & Management	Allow HR personnel to create job descriptions with screening criteria	UC-07

FR-ID	Feature Area	Requirement	Use Case
FR12	Job Creation & Management	Allow HR personnel to edit existing job postings	UC-08
FR13	Resume Viewing & Screening	Allow HR personnel to view and read extracted resume content	UC-12
FR14	Resume Viewing & Screening	Allow HR personnel to shortlist candidates based on job relevance	UC-12
FR15	Resume Viewing & Screening	Allow HR personnel to reject candidates	UC-12
FR16	Resume Viewing & Screening	Allow HR personnel to export shortlisted candidates to a spreadsheet	UC-06
FR17	Email Integration & Resume Sync	Allow HR personnel to connect one or more email accounts	UC-09
FR18	Email Integration & Resume Sync	Sync resume attachments from connected email accounts	UC-09
FR19	Logs & Metadata Management	Allow users to view activity logs	UC-10
FR20	Logs & Metadata Management	Allow users to view extracted document metadata (skills, experience)	UC-11

FR-ID	Feature Area	Requirement	Use Case
FR21	Search & Retrieval	Keep question-and-answer threads so that follow-up questions can be understood in context	UC-04
FR22	Email Integration & Resume Sync	Optionally remove documents when their source email is permanently deleted in Gmail	UC-09

Beyond the numbered requirements, the built system adds two behaviours. A RAG question-answering capability returns AI-generated answers with citations to the source documents (`/rag/query` and `/rag/stream`). A candidate-matching capability scores and ranks candidates against a job on skill overlap, experience fit, and vector similarity (`POST /jobs/{id}/match`). Both extend FR07-FR09 (search) and FR13-FR15 (screening) into the semantic and ranking domains that the project’s RAG focus promised.

3.3 Non-Functional Requirements

The non-functional requirements define the quality attributes the system must satisfy. They derive from the Design and Implementation Constraints, Operating Environment, and Assumptions sections of the SRS, and the implemented architecture reflects them.

3.3.1 Performance

Document ingestion, covering extraction, classification, chunking, embedding, and indexing, runs asynchronously on Celery workers. Uploads return at once and the interface stays responsive. Search merges four retrieval signals, vector similarity, lexical full-text search, structured SQL metadata filtering, and fuzzy matching, through Reciprocal Rank Fusion, and RAG answers stream token-by-token to cut perceived latency. The target hardware profile is modest: 16 GB RAM and a quad-core processor, with SSD storage preferred.

3.3.2 Security and Privacy

Authentication uses JWT [18] access and refresh tokens with Argon2id [19] password hashing, timing-safe verification, refresh-token rotation, and a Redis denylist for revoked tokens. Password-reset tokens are single-use, hashed, and short-lived. Role-based access control across HR and admin roles restricts functionality, and every

data-access path is scoped by owner. In production the data stores sit on a private Docker network with only the nginx port exposed, and client-server traffic runs over HTTPS. The SRS also mandates encryption at rest and in transit, audit trails, and compliance with data-protection regulations such as GDPR and CCPA, depending on where the system is deployed.

3.3.3 Scalability

The stateless FastAPI backend and the separate Celery worker pool scale horizontally behind nginx. ChromaDB holds vector embeddings per model, PostgreSQL provides indexed metadata storage, and MinIO offers S3-compatible object storage that can give way to a cloud object store. Because the adapter design is modular and Protocol-based, components such as the LLM, OCR, embeddings, and storage can each be replaced or scaled on their own.

3.3.4 Usability

The system ships as a browser-based single-page application (React 19, Tailwind v4, shadcn/ui) that needs no client-side installation and runs on modern browsers including Chrome, Firefox, Edge, and Safari. Consistent navigation, natural-language search, toast notifications, loading indicators, pagination, and automatic session-timeout handling all help HR users of moderate technical skill.

3.3.5 Reliability

Background tasks use `acks_late` with three delayed retries, and an indexing failure is treated as non-fatal, so a document stays usable even when one processing step fails. Version stamps on the extraction, chunking, and embedding steps make targeted re-indexing and recovery possible. Duplicate checks guard the resume-sync pipeline.

3.3.6 Maintainability

The codebase enforces a strict layered architecture with explicit import rules. Business logic stays free of framework and infrastructure concerns, and every failure travels through domain exceptions that map centrally to HTTP status codes. A self-documenting OpenAPI specification generates the frontend SDK, linting runs before every commit, and the acceptance bar requires services to be testable with fakes and no Docker.

3.4 Tools and Technologies Used

The tools and technologies below were chosen to implement Hireflow. The selection favours open-source, modular components that meet the SRS constraints of CPU-or-GPU deployment and reduced vendor lock-in.

Table 3.2: Tools and Technologies

Category	Tool / Technology	Purpose
Backend Framework	FastAPI (Python 3.12, uv) [20]	Asynchronous REST API with automatic OpenAPI generation
Data Validation	Pydantic v2	Request/response schemas and typed settings management
ORM & Migrations	SQLAlchemy 2, Alembic	Relational data access and async schema migrations
Relational Database	PostgreSQL 15 [21]	Users, documents, jobs, candidates, applications, activity logs, full-text search
Vector Database	ChromaDB [22]	Storage and cosine similarity search over chunk embeddings
Task Queue / Broker	Celery [23] + Redis 7 [24]	Background document processing and token denylist/reset stores
Object Storage	MinIO (S3-compatible) [25]	Storage of original uploaded files
Embeddings	sentence-transformers (bge-small-en-v1.5 [26], swappable)	Semantic vector representations of document chunks
Large Language Model	Anthropic Claude (Ollama fallback)	RAG question answering and LLM-based classification fallback
Document Parsing	PyMuPDF, unstructured	Text and structured element extraction from PDF/Word
OCR	Tesseract / pytesseract (Claude/Ollama vision optional)	Text extraction from scanned images
Authentication	JWT, Argon2 (argon2id)	Token-based auth and secure password hashing

Category	Tool / Technology	Purpose
Frontend Framework	React 19 [27] + TypeScript	Single-page web application
Build Tooling	Vite	Frontend build and dev server
Styling / UI	Tailwind CSS v4, shadcn/ui	Component styling and design system
Data Fetching	TanStack Query	Server-state management with generated query/mutation hooks
Routing & Charts	react-router, recharts	Client-side routing and dashboard visualisations
API Client	OpenAPI to hey-api/openapi-ts	Type-safe generated SDK from the backend spec
Reverse Proxy	nginx	Serves frontend static assets and proxies <code>/api/</code> in production
Containerisation	Docker, Docker Compose [28]	Reproducible dev and production deployment
Local Orchestration	Make, Tilt	One-command bring-up of the API, worker, frontend, and backing services
Version Control	Git, GitHub	Source control and issue tracking for the feature-per-iteration workflow
Testing	pytest, pytest-asyncio	Unit, integration, and API test suites
Code Quality	ruff	Linting and formatting, run as a gate before a change is committed
Diagramming	draw.io	UML, data-flow, and entity-relationship figures in this report

3.5 Use Cases / User Stories

Twelve use cases model the system's behaviour, spanning authentication, document upload and handling, search, job management, screening, email integration, and auditing. HR Personnel are the primary actor in most of them. The Email Service acts as an external actor for resume intake and synchronisation. Figure 3.1 presents the overall use case diagram.

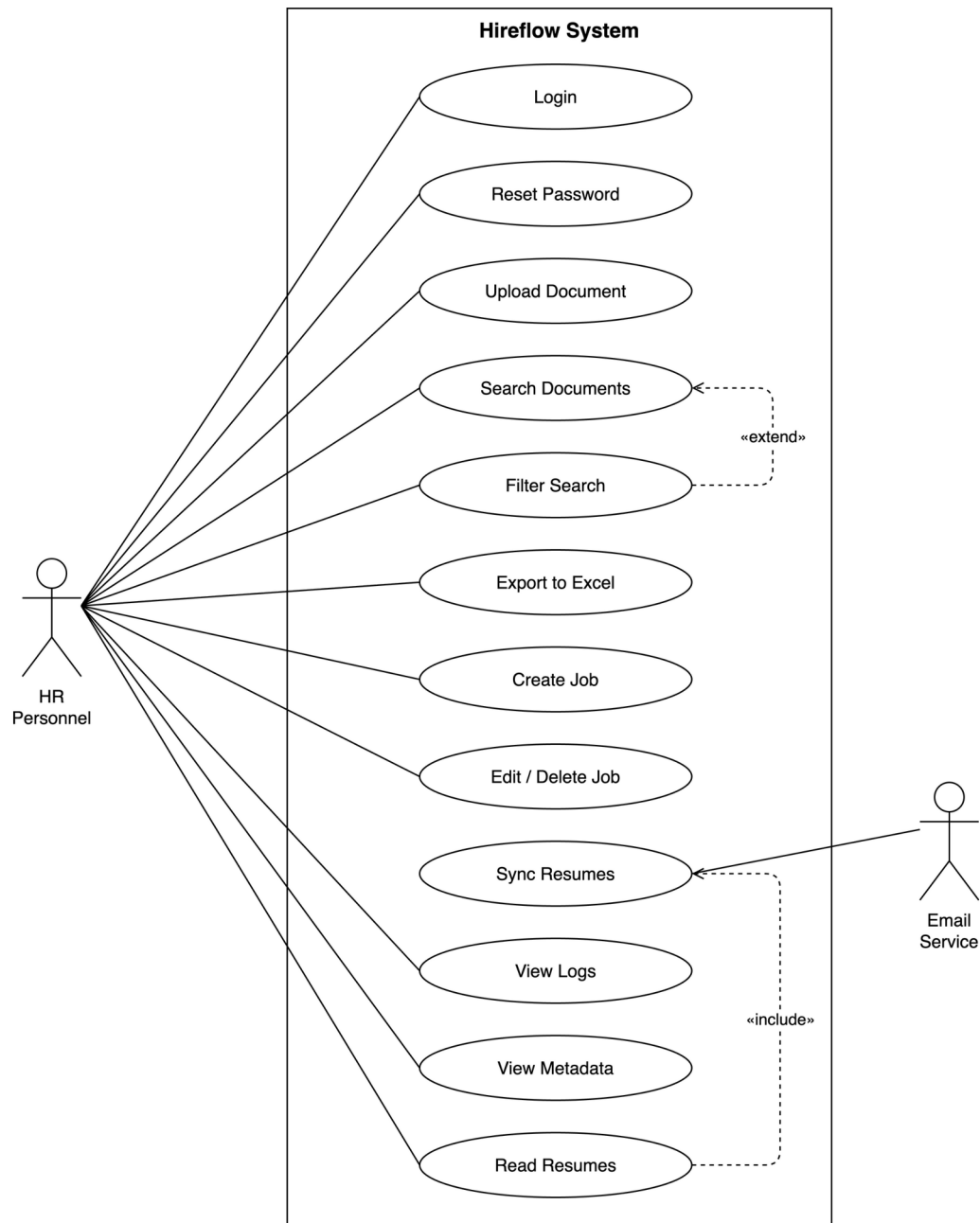


Figure 3.1: Use Case Diagram

The main use cases identified in the SRS are summarised below.

Table 3.3: Summary of Use Cases (UC-01-UC-12)

Use Case	Name	Primary Actor	Summary
UC-01	Login	HR Personnel	Authenticate with email and password to access the system
UC-02	Reset Password	HR Personnel	Recover access to a forgotten account via a verification mechanism
UC-03	Upload Document	HR Personnel	Upload resumes and HR documents for text extraction, indexing, and storage
UC-04	Search Documents	HR Personnel	Search stored documents and view ranked results
UC-05	Filter Search	HR Personnel	Refine search results using metadata filters
UC-06	Export to Excel	HR Personnel	Export results or shortlisted candidates to a spreadsheet
UC-07	Create Job	HR Personnel	Create a job posting with screening criteria
UC-08	Edit/Delete Job	HR Personnel	Update or remove an existing job posting
UC-09	Sync Resumes	Email Service	Fetch and process resume attachments from connected accounts
UC-10	View Logs	HR Personnel	Review system

Use Case	Name	Primary Actor	Summary
			activity and audit trail
UC-11	View Metadata	HR Personnel	View extracted document metadata such as skills and experience
UC-12	Read Resumes	HR Personnel	Read extracted resume content and shortlist or reject candidates

User Stories. The following user stories capture representative goals from the HR user's perspective:

- **US-1 (Search):** As an HR user, I want to ask questions about my documents in plain English so that I can find relevant information without remembering exact keywords or filenames.
- **US-2 (Screening):** As an HR user, I want the system to rank incoming resumes against a job's required skills and experience so that I can focus on the most relevant candidates first.
- **US-3 (Email Sync):** As an HR user, I want resumes to be collected automatically from my connected Gmail account so that I do not have to download and upload attachments manually.
- **US-4 (Auditing):** As an HR user, I want to view an activity log of uploads, searches, and screening actions so that I can audit what happened and when.

3.6 Use Cases Table

To illustrate the full structure of a use case, the User Login flow (UC-01) is documented in detail below, based on the system's actual JWT-based authentication mechanism.

Table 3.4: Sample Use Case, User Login

Field	Description
Use Case ID	UC-01
Use Case Name	User Login
Primary Actor	Registered HR User
Goal	Authenticate with valid credentials to

Field	Description
	obtain an authorised session and access protected system features
Preconditions	1. The system is running and reachable. 2. The user has a registered, active account.
Trigger	The user selects “Sign in” on the login page after entering their credentials.
Main Flow	1. The user opens the login page. 2. The user enters their email and password. 3. The system verifies the account exists and is active. 4. The system verifies the password against the stored Argon2id hash using timing-safe comparison. 5. The system issues a short-lived access token (30 min) and a refresh token (7 days). 6. The system records the login in the activity log. 7. The user is redirected to the dashboard.
Alternative Flow(s)	3a/4a. If the credentials are invalid, the system rejects the attempt and returns an authentication error without revealing whether the email or password was wrong. 2a. If the user has forgotten their password, they follow the “Forgot password?” link, invoking UC-02 (Reset Password).
Postconditions	The user holds a valid access/refresh token pair and an authenticated session; the login event is written to the audit trail.
Exceptions	Account disabled leads to access forbidden (403). A malformed or expired token on subsequent requests leads to 401 with a prompt to re-authenticate.
Related Functional Requirement(s)	FR01, FR03

3.7 Development Methodology

Hireflow was built with an incremental, feature-driven process rather than a single-pass waterfall. Scope was fixed early by the SRS, which bounded the work at twenty-two functional requirements and twelve use cases, but the system itself was delivered one tracked feature at a time. Each iteration carried a single feature from implementation through its own tests before the next was started, so a working subset of the platform existed throughout, and integration problems surfaced early instead of piling up at the end. Figure 3.2 sets out the flow.

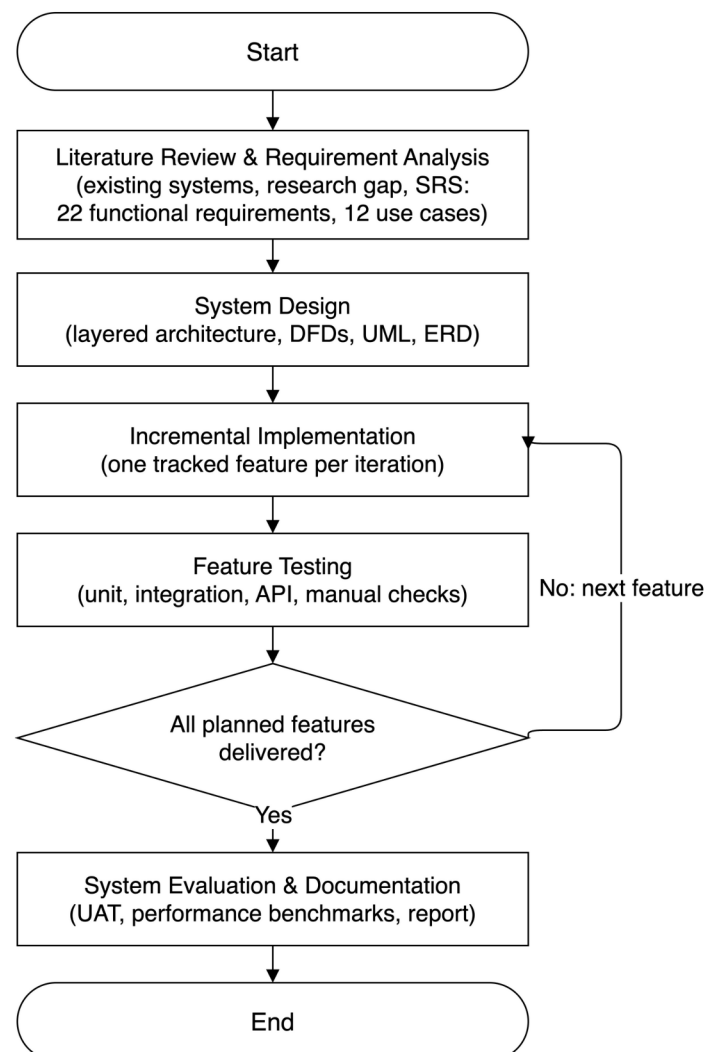


Figure 3.2: Development Methodology Flowchart

The process opens with a literature review and requirement analysis, which together establish the research gap and produce the SRS. System design follows, fixing the layered architecture, the data flow diagrams, the UML models, and the relational schema before any feature is coded. The implementation stage is the iterative core: a feature is built, then exercised by unit, integration, and API tests along with manual checks, and the decision point asks whether every planned feature has been delivered. While features remain, control returns to implementation and the next one begins; that

loop is the difference between an incremental model and a sequential one. Once the feature set is complete the system moves to full-system evaluation, covering user-acceptance testing and the performance benchmarks reported in Chapter 6, and finally to documentation.

CHAPTER 4: SYSTEM DESIGN

This chapter turns the requirements set out earlier into a concrete technical design for Hireflow, an AI-powered HR screening and document-retrieval platform. It presents the system's layered architecture, the flow of data through its ingestion and retrieval pipelines, the structural and behavioural UML models, the relational database schema derived from the implemented data model, and the user-interface design. Each artefact is grounded in the system as actually built. The backend is a FastAPI application organised into strict architectural layers; document processing runs asynchronously on Celery; a hybrid Retrieval-Augmented Generation (RAG) subsystem spans PostgreSQL and ChromaDB; and the frontend is a React single-page application. Between them, these views describe how the system divides responsibilities, enforces access control, and answers natural-language questions with verifiable citations over an organisation's unstructured documents.

4.1 System Architecture Description & Design

Hireflow follows a layered, service-oriented architecture that separates presentation, application coordination, domain logic, and infrastructure. The design goal is straightforward: business rules should be testable in isolation, infrastructure providers should be swappable, and long-running work should never block the request path.

At the outermost tier, a React 19 single-page application (built with TypeScript, Vite, Tailwind CSS v4 and shadcn/ui) runs in the browser and communicates exclusively through a generated, type-safe SDK. The SDK is produced from the backend's OpenAPI specification via `openapi-ts`, so every request and response stays contractually aligned with the server. In production, an nginx reverse proxy serves the compiled static frontend and forwards `/api/` traffic to the backend. The data stores remain on a private Docker network with no host-exposed ports.

The FastAPI API layer is the system's HTTP boundary. Routes are kept thin, five lines or fewer per handler: they validate requests using Pydantic v2 schemas, invoke a service, and serialise the result. Shared concerns are handled here through dependency injection, covering authentication (`CurrentUser`), role gating (`RequireAdmin`), and a central error handler that maps domain exceptions to HTTP status codes. Routes never raise `HTTPException` directly. Instead, services raise typed `DomainError` subclasses (for example `NotFound`, `Forbidden`, `FileTooLarge`) that the handler translates. The public surface comprises 31 endpoints grouped by tag: authentication, documents, search, RAG, jobs, candidates, users, and activity logs.

Beneath the API sits the service layer, the application's coordination tier. Around ten services (authentication, document, search, RAG, job, candidate, matching, activity, and others) orchestrate repositories and adapters to fulfil use cases. Services depend only on abstractions: repositories for data access and *Protocol*-typed adapters for infrastructure. That is why the system is testable. A service can be instantiated in a unit test with in-memory fakes, needing neither a database nor Docker. The domain

layer underneath holds pure business rules, such as authorization checks and exception definitions, and imports nothing from infrastructure.

Infrastructure is reached through the adapter layer. Each external capability is defined as a Protocol with one or more concrete implementations, which yields runtime-swappable providers: `BlobStorage` (MinIO, replaceable by S3 or GCS), `VectorStore` (ChromaDB), `LlmProvider` (Anthropic Claude or Ollama), `EmbeddingProvider` (sentence-transformers, default BAAI/bge-small-en-v1.5), `VisionProvider` for OCR (Claude, Ollama, or Tesseract), `PasswordHasher` (Argon2id), and `TokenIssuer` (JWT). The concrete graph is wired once in a composition root (`api/deps.py`), where heavyweight resources such as the embedding model and vector store are constructed as process singletons.

The supporting infrastructure comprises PostgreSQL 15 (relational data, plus a generated `tsvector` column powering lexical full-text search), ChromaDB (vector embeddings for semantic search), Redis 7 (Celery broker, JWT revocation denylist, and one-time password-reset tokens), MinIO (S3-compatible object storage for uploaded files), and the configured LLM provider (Claude) for answer generation and classification fallback.

Two pipelines dominate the system's behaviour. The ingestion pipeline runs asynchronously in a Celery worker. An upload returns HTTP 201 immediately with `status=PENDING`; the worker then fetches the blob from MinIO, extracts layout-aware typed elements (via the `unstructured` library, PyMuPDF, and Tesseract OCR for scanned images), classifies the document, and chunks it with heading- and table-aware rules. Each chunk is contextualised, embedded, and upserted into ChromaDB, while PostgreSQL's search vector auto-populates. On completion the document reaches `status=READY`, at which point an on-ready hook can auto-create a candidate from a resume. The RAG pipeline runs inline per request. It retrieves the most relevant chunks (owner-scoped, READY-only), classifies the question's intent, composes a layered system prompt, and streams the answer from Claude with typed citations. Both pipelines and their interactions are depicted in Figure 4.1.

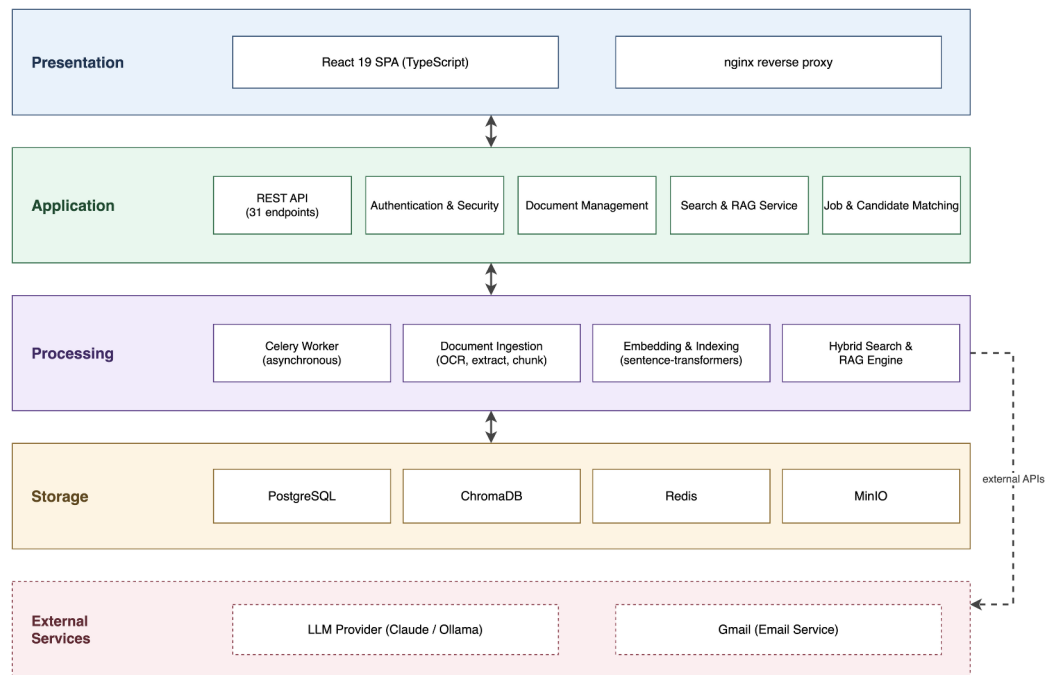


Figure 4.1: System Architecture

4.2 Data Flow Diagrams

Data Flow Diagrams (DFDs) model the system as a network of processes that transform data, the external entities that supply and consume it, and the data stores that persist it. They are presented in progressively decomposed levels. Level 0, the context diagram, treats the entire system as a single process to fix its boundary and external interactions. Level 1 decomposes that process into the major functional subsystems. Level 2 then expands a chosen subsystem into its constituent steps.

The Level 0 (context) diagram situates Hireflow as one process bounded by its external actors. The primary external entity is the HR user (and the privileged Administrator), who uploads documents, issues search and natural-language queries, manages jobs and candidates, and receives ranked results and cited answers. Two further external entities appear: the Gmail service, from which candidate emails and attachments are synced, and the LLM provider (Anthropic Claude), which receives composed prompts and returns generated answers. This level establishes what crosses the system boundary without revealing internal structure, as shown in Figure 4.2.

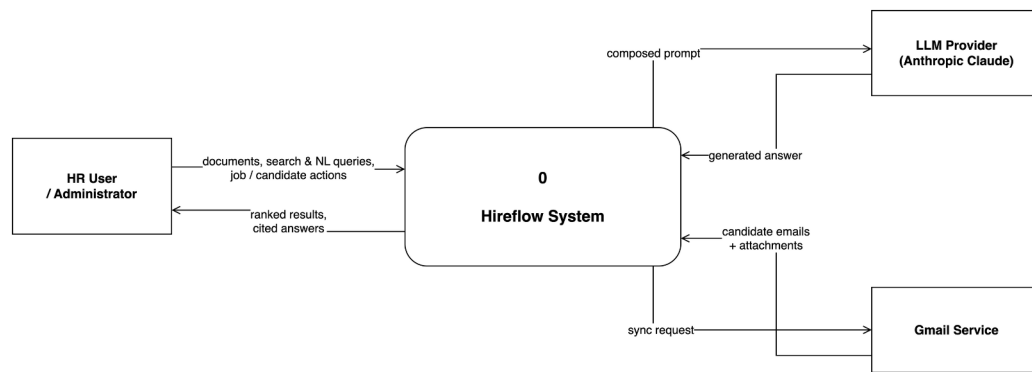


Figure 4.2: DFD Level 0 (Context Diagram)

The Level 1 diagram decomposes the single context process into the system's principal subsystems and the data stores between them: authentication and session management; document ingestion and storage; the hybrid search and RAG engine; job and candidate management with resume-to-job matching; Gmail synchronisation; and activity logging. It shows how an uploaded file flows from the API into object storage and the asynchronous processing pipeline, how extracted text and metadata land in the relational and vector stores, and how a query fans out across those stores before results are fused and returned. The major data stores are made explicit here: Users, Documents, Candidates, Jobs, Applications, the vector collection, and the activity log (Figure 4.3).

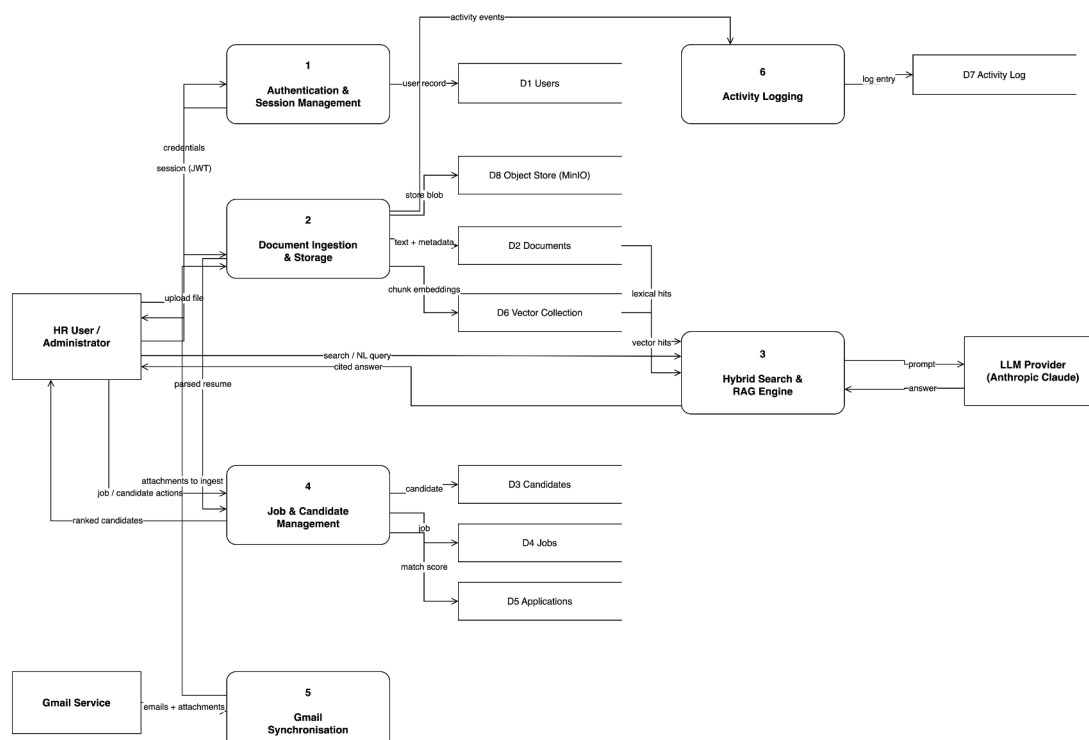


Figure 4.3: DFD Level 1

The Level 2 diagram drills into the document-processing and retrieval subsystem, the most data-intensive part of the platform. It expands both the ingestion pipeline and the

query pipeline into their constituent processes, exposing how chunks, embeddings, and ranked hits move between the worker, PostgreSQL, ChromaDB, and the LLM provider. The individual ingestion stages are documented in Table 5.2 (Figure 4.4).

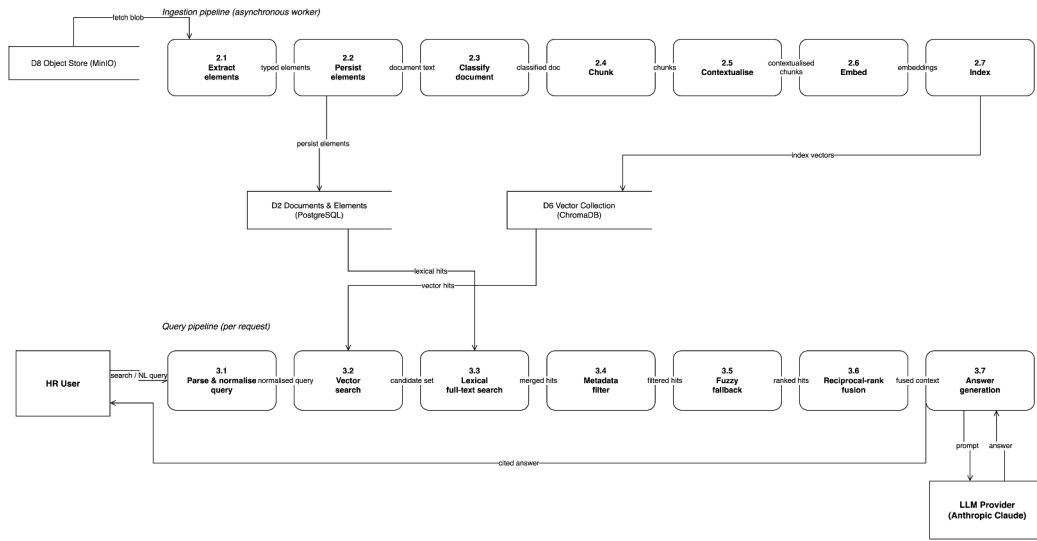


Figure 4.4: DFD Level 2

4.3 UML Diagrams

The Unified Modeling Language (UML) captures the system's static structure and dynamic behaviour. The class, activity, component, and sequence views of Hireflow are presented below.

4.3.1 Class Diagram

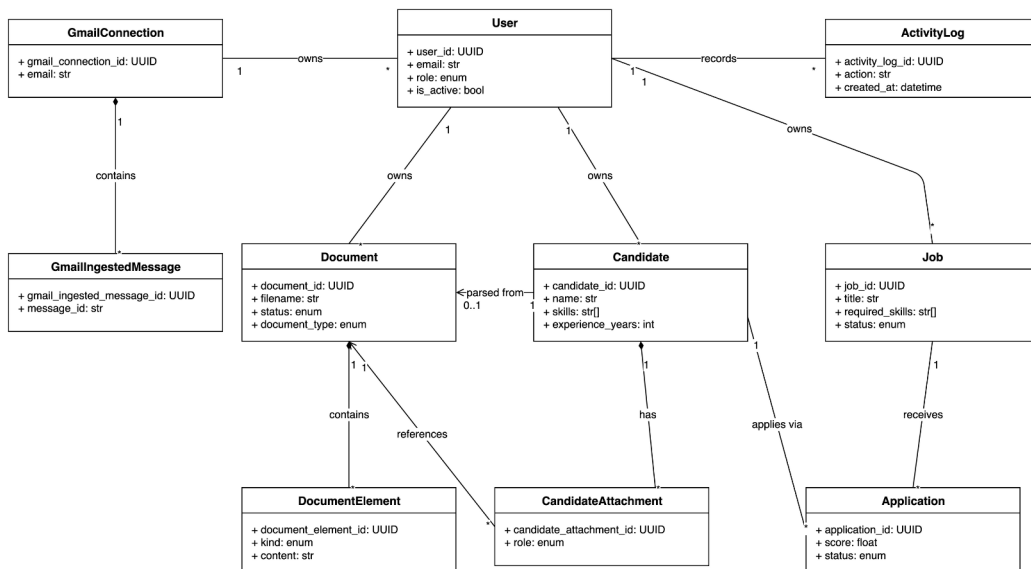


Figure 4.5: Class Diagram

4.3.2 Activity Diagram

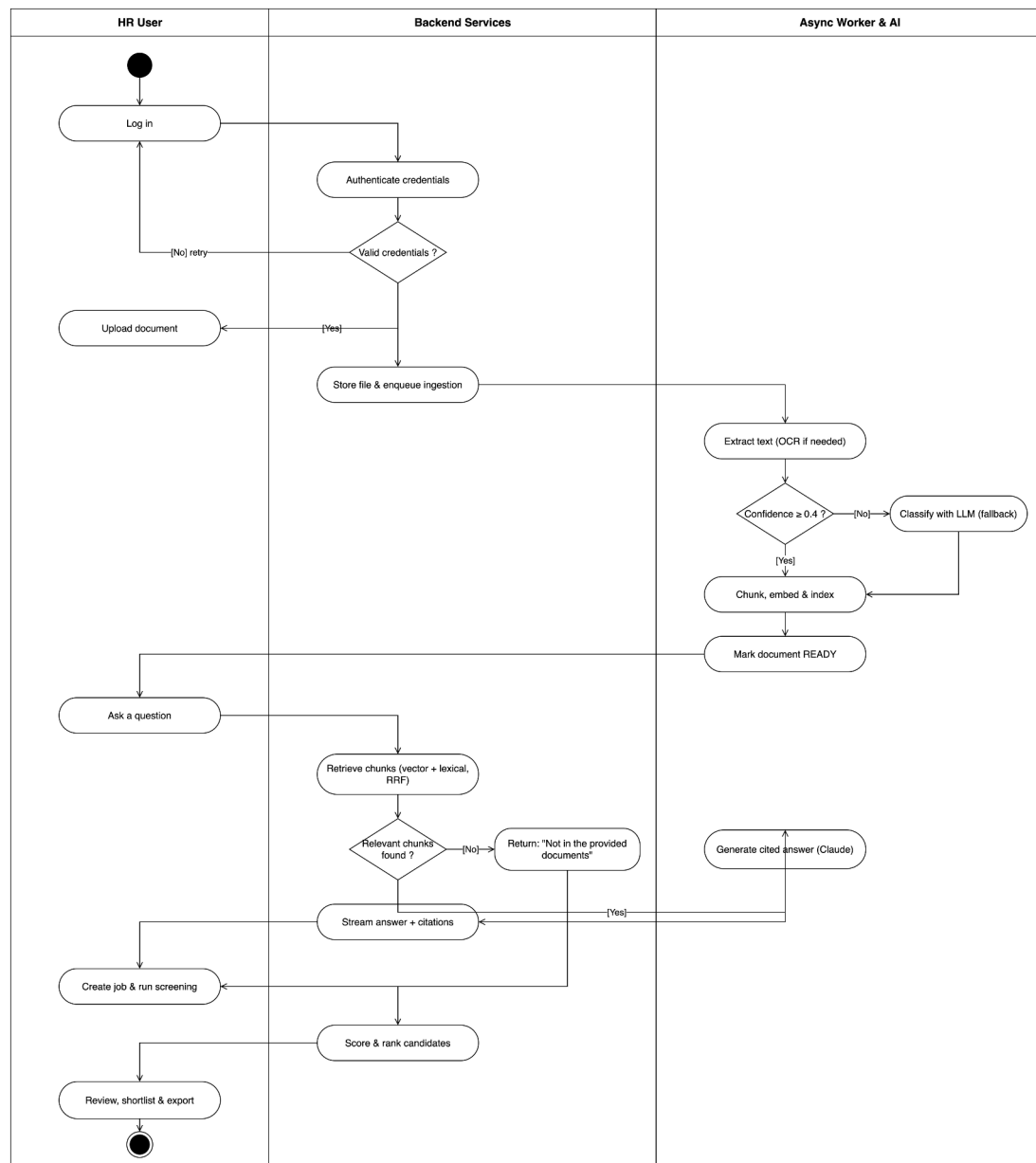


Figure 4.6: Activity Diagram

4.3.3 Component Diagram

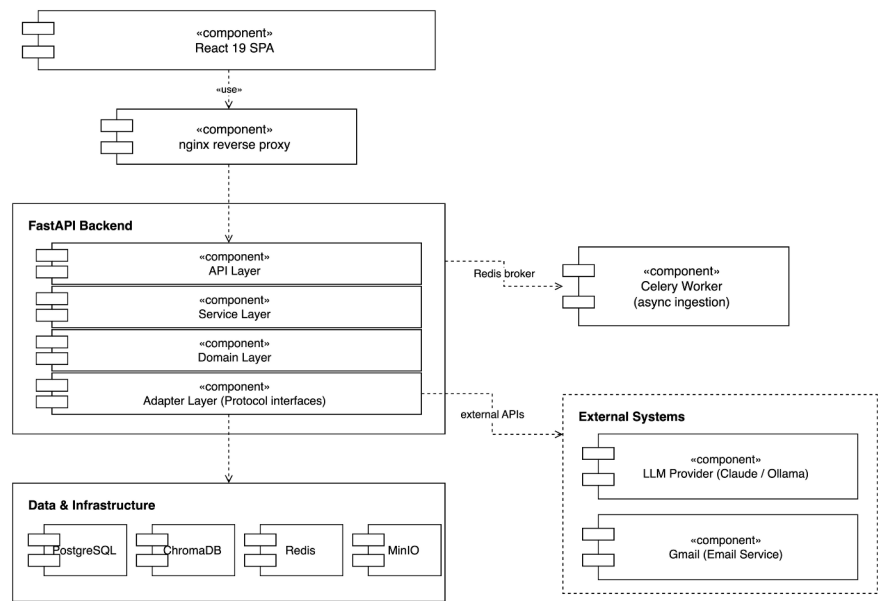


Figure 4.7: Component Diagram

4.3.4 Sequence Diagram

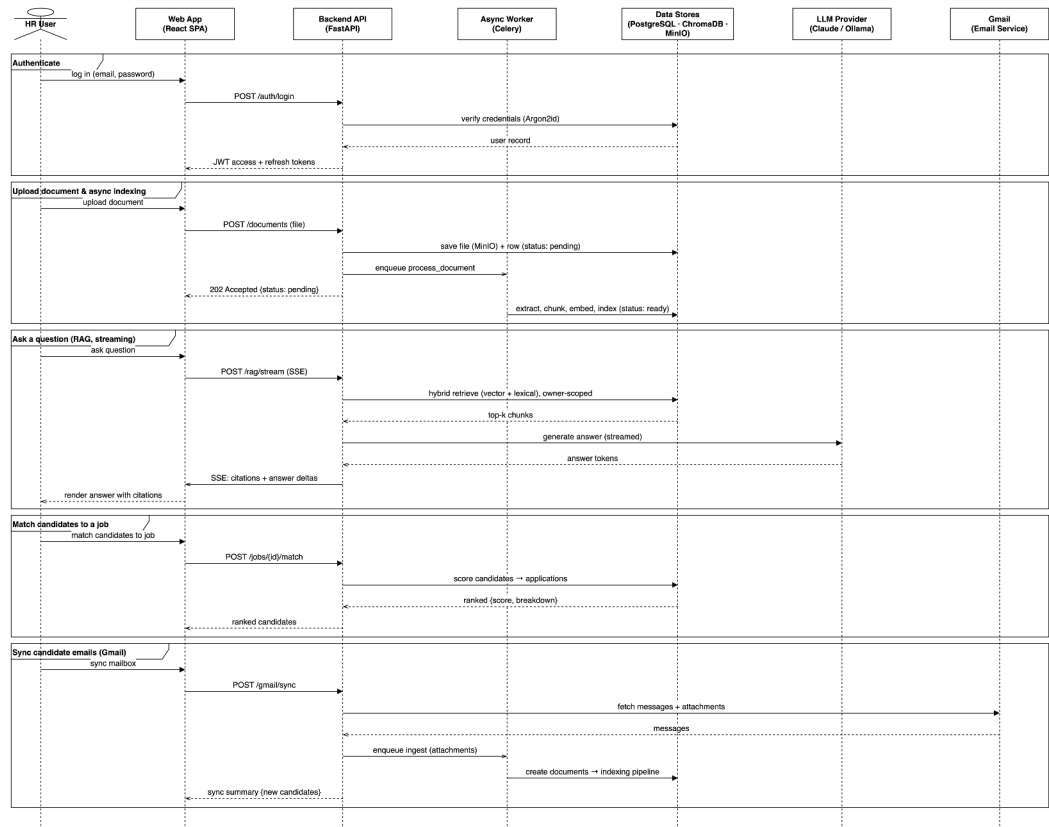


Figure 4.8: Sequence Diagram (Streaming RAG Query)

4.3.5 State Machine Diagram

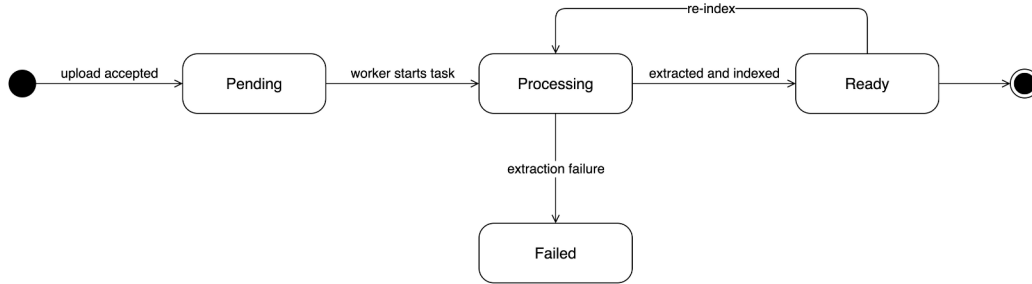


Figure 4.9: Document Status State Machine

4.4 Database Design (ERD, Schema, Data Dictionary)

The persistent state of Hireflow lives in a PostgreSQL 15 relational schema, complemented by a ChromaDB vector store for embeddings. This section describes the entity relationships, provides a schema overview, and documents the core tables as data dictionaries. The relational design derives directly from the implemented SQLAlchemy models.

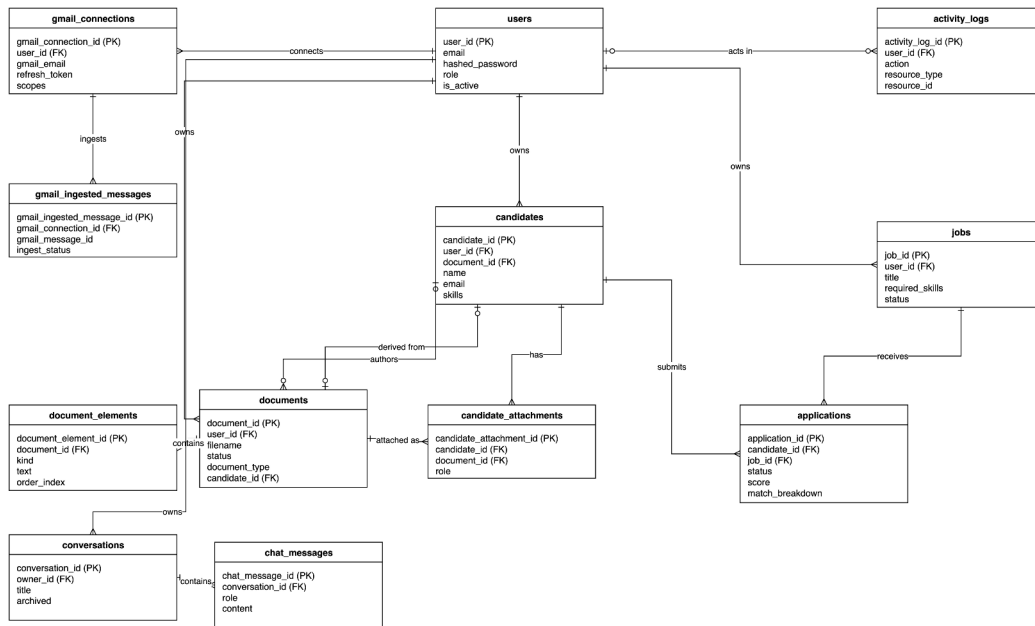


Figure 4.10: Entity-Relationship Diagram

Schema overview. Every table has a UUID primary key named after the table (for example `user_id`, `document_id`) and `created_at` / `updated_at` timestamp columns from shared mixins. PostgreSQL-native features are used throughout: enumerated types (`user_role`, `document_status`, `document_type`, `job_status`, `application_status`, `attachment_role`, and others), array columns (`required_skills`, `skills`, `education`, `scopes`), a JSONB metadata column on documents, and a database-generated, weighted `tsvector` column (`search_tsv`) that indexes filename (weight A), skills (weight B), and body text

(weight C) for lexical retrieval via a GIN index. Sensitive fields such as `full_name`, `phone`, and `Gmail refresh_token` are stored using an encrypted column type. The four core tables are documented below.

Table 4.1: users Data Dictionary

Field	Type	Constraints	Description
<code>user_id</code>	UUID	PK	Unique identifier
<code>email</code>	VARCHAR(320)	UNIQUE, NOT NULL, INDEX	Login email address
<code>hashed_password</code>	VARCHAR(255)	NOT NULL	Argon2id password hash
<code>full_name</code>	Encrypted string	NULLABLE	User's display name (encrypted at rest)
<code>role</code>	ENUM(hr, admin)	NOT NULL, DEFAULT hr	Authorisation role
<code>is_active</code>	BOOLEAN	NOT NULL, DEFAULT true	Whether the account is enabled
<code>created_at</code>	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Creation timestamp
<code>updated_at</code>	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Last-modified timestamp

Table 4.2: documents Data Dictionary

Field	Type	Constraints	Description
<code>document_id</code>	UUID	PK	Unique identifier
<code>user_id</code>	UUID	FK → users.user_id, ON DELETE CASCADE, NOT NULL, INDEX	Owning HR user
<code>filename</code>	VARCHAR(512)	NOT NULL	Original uploaded file name
<code>mime_type</code>	VARCHAR(128)	NOT NULL	Detected MIME type
<code>size_bytes</code>	BIGINT	NOT NULL	File size in bytes

Field	Type	Constraints	Description
storage_key	VARCHAR(1024)	UNIQUE, NOT NULL	MinIO object key
status	ENUM(pending, processing, ready, failed)	NOT NULL, DEFAULT pending, INDEX	Processing lifecycle state
document_type	ENUM(resume, report, contract, letter, other)	NULLABLE	Classified document type
extracted_text	TEXT	NULLABLE	Full extracted body text
metadata	JSONB	NULLABLE	Extracted metadata (skills, experience, etc.)
extraction_version	VARCHAR(32)	NULLABLE	Extractor version stamp for re-indexing
chunking_version	VARCHAR(32)	NULLABLE	Chunking-rules version stamp
embedding_model_version	VARCHAR(128)	NULLABLE	Embedding model used at last index
candidate_id	UUID	FK → candidates.candidate_id, ON DELETE SET NULL, NULLABLE, INDEX	Linked authoring candidate
search_tsv	TSVECTOR	GENERATED, GIN INDEX	Weighted full-text search vector
created_at / updated_at	TIMESTAMPTZ	NOT NULL	Audit timestamps

Table 4.3: jobs Data Dictionary

Field	Type	Constraints	Description
job_id	UUID	PK	Unique identifier

Field	Type	Constraints	Description
user_id	UUID	FK → users.user_id, ON DELETE CASCADE, NOT NULL, INDEX	Owning HR user
title	VARCHAR(255)	NOT NULL	Job title
description	TEXT	NOT NULL	Full job description
required_skills	VARCHAR[]	NOT NULL, DEFAULT {}	Mandatory skills
preferred_skills	VARCHAR[]	NULLABLE	Desirable skills
education_level	VARCHAR(100)	NULLABLE	Minimum education requirement
experience_min	INTEGER	NOT NULL, DEFAULT 0	Minimum years of experience
experience_max	INTEGER	NULLABLE	Maximum years of experience
location	VARCHAR(255)	NULLABLE	Job location
status	ENUM(draft, open, closed, archived)	NOT NULL, DEFAULT draft, INDEX	Job posting state
created_at / updated_at	TIMESTAMPTZ	NOT NULL	Audit timestamps

Table 4.4: candidates Data Dictionary

Field	Type	Constraints	Description
candidate_id	UUID	PK	Unique identifier
user_id	UUID	FK → users.user_id, ON DELETE CASCADE, NOT NULL, INDEX	Owning HR user
document_id	UUID	FK →	Resume the

Field	Type	Constraints	Description
		documents.document_id, ON DELETE SET NULL, UNIQUE, NULLABLE, INDEX	candidate was parsed from
name	VARCHAR(255)	NULLABLE	Candidate name
email	VARCHAR(320)	NULLABLE, partial UNIQUE per owner	Candidate email
phone	Encrypted string	NULLABLE	Candidate phone (encrypted at rest)
skills	VARCHAR[]	NOT NULL, DEFAULT {}	Extracted skills
experience_years	INTEGER	NULLABLE	Total years of experience
education	VARCHAR[]	NULLABLE	Education history entries
supplementary_keywords	VARCHAR[]	NULLABLE	Keywords from non-resume attachments
summary	VARCHAR(1024)	NULLABLE	One-sentence recruiter brief
created_at / updated_at	TIMESTAMPTZ	NOT NULL	Audit timestamps

Alongside the relational schema, ChromaDB holds one collection per embedding model (named `documents_<model_slug>`). Each stored chunk is keyed as `<document_id>:<chunk_index>` and carries its embedding vector, the chunk text, and metadata (`document_id`, `user_id`, `chunk_index`, `chunk_kind`, `section_heading`, `page_number`, `chunking_version`, plus flattened document-level fields such as `filename` and `document_type`). This denormalised metadata enables owner-scoped, type-filtered vector retrieval without a round-trip to PostgreSQL.

4.5 UI/UX Design

The user interface is a React 19 single-page application written in TypeScript and built with Vite. Its design philosophy is a clean, professional, information-dense workspace suited to HR personnel who work with large document sets and need results at a glance. The visual layer is built on Tailwind CSS v4 utility classes composed into accessible shadcn/ui primitives (backed by Base UI), which gives a consistent component vocabulary across every screen: cards, dialogs, tables, badges, tooltips, and scroll areas. The application supports light and dark theming. All data-bearing components consume the auto-generated, type-safe API SDK through TanStack Query, so loading, empty, and error states are handled uniformly and no screen relies on mock data.

Navigation is organised around the primary HR workflows, each a dedicated page wired to the real API: a Dashboard overview, Documents management (upload, list, in-app preview, metadata, and delete), Search (combining hybrid keyword/semantic search with a streaming RAG chat), Jobs (create, edit, and match candidates), Candidates (a screening list with match scores), Logs (the activity audit trail), and Settings (profile and password). Authentication is handled transparently. An `AuthProvider` hydrates the current user on mount, and a client interceptor attaches the bearer token and silently refreshes it on expiry via a single-flight retry, so the user is never spuriously logged out mid-task.

Two screens embody the system's differentiating design work. The Documents page provides drag-and-drop upload with real-time status badges that follow the document status lifecycle shown in Figure 4.9, giving users immediate feedback while indexing happens out-of-band. The Search page implements a conversational layout: the input is pinned to the bottom of the card, messages scroll within a fixed viewport, and assistant answers render as GitHub-flavoured Markdown (tables, lists, and emphasis) with a typing indicator and blinking cursor during streaming. Most importantly, inline citation chips are parsed from the answer text and matched against source documents. Hovering a chip reveals the filename, section heading, and snippet; clicking it scrolls the corresponding source card into view with a highlight flash, making every generated claim traceable to its evidence. Representative screens are shown in Figures 4.11 through 4.19.

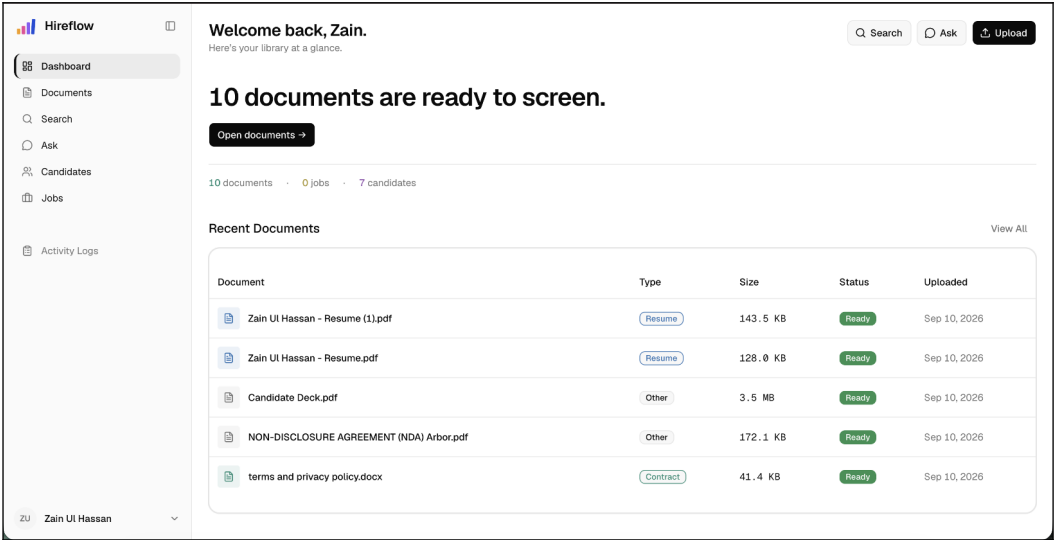


Figure 4.11: Dashboard overview

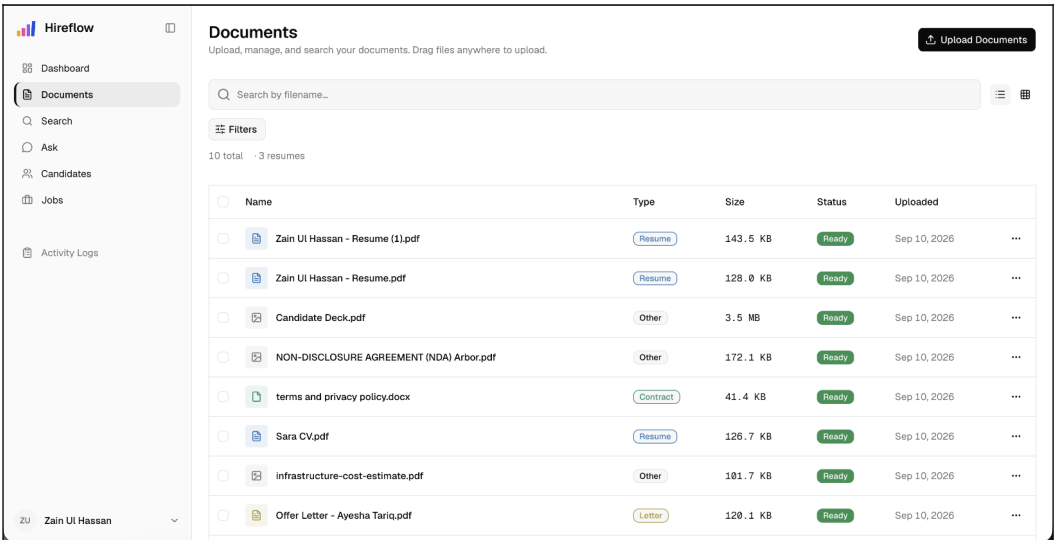


Figure 4.12: Documents management with upload and processing status

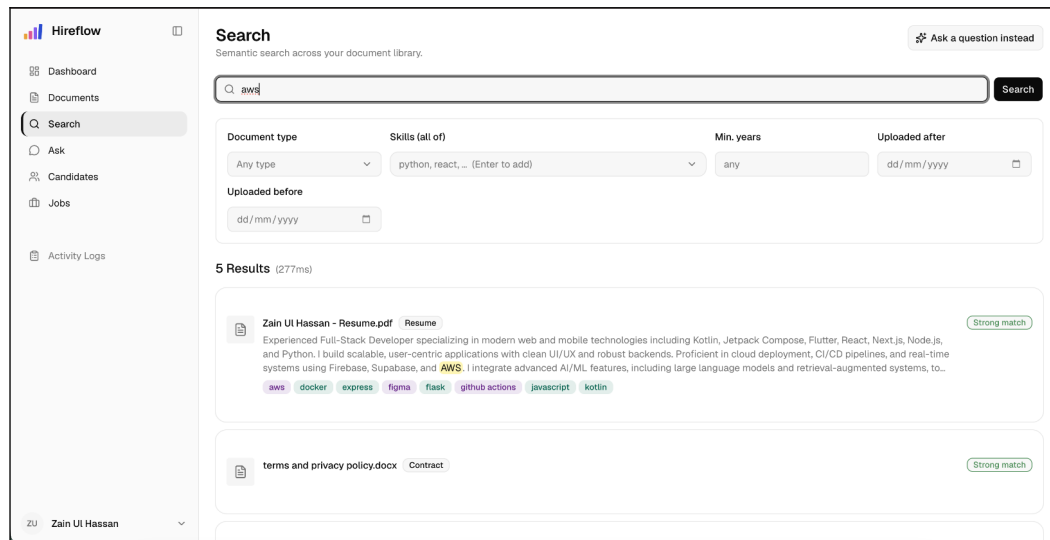


Figure 4.13: Semantic search across the document library with match scores and skill filters

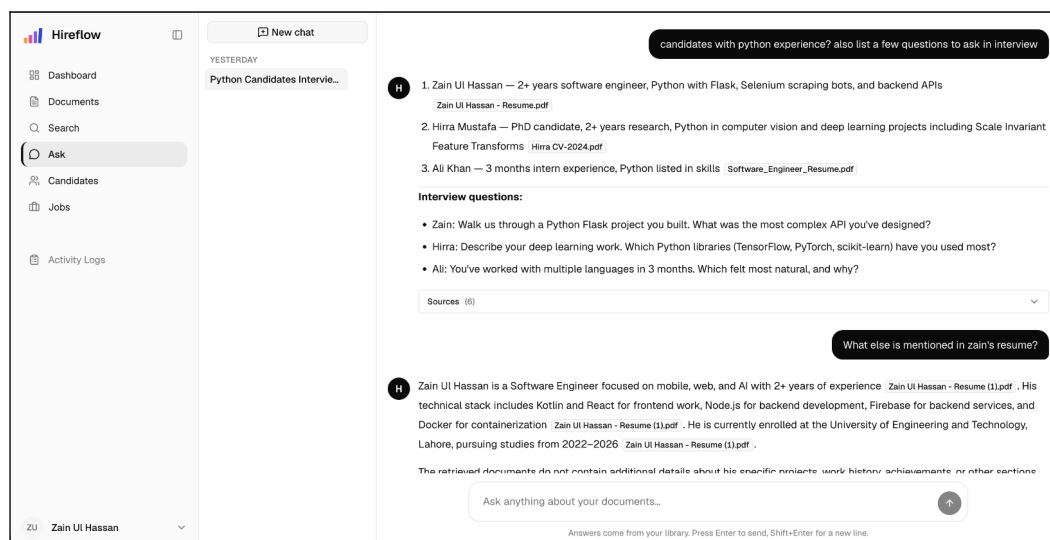


Figure 4.14: Ask Hireflow, conversational question answering grounded in the library with source citations

Hireflow

Dashboard

Documents

Search

Ask

Candidates

Jobs

Activity Logs

Candidates

Candidates extracted from uploaded resumes

Search by name, email, or skill...

Name	Email	Skills	Experience	Education
—	—	—	—	BS, ME
—	zainulhassan4330@gmail.com	aws docker express figma flask +9	—	Bachelor
—	hirramustafa@gmail.com	computer vision deep learning jira machine learning python	—	BS, Diploma, MS, Masters, PhD, PhD, be
—	all.khan@gmail.com	c++ css html javascript python	—	BS
—	noor.uet.11@gmail.com	.net c# c++ css html +2	—	MS
—	kshoab@yahoo.com	c++ computer vision java machine learning matlab	—	Associate, BS, MS, PHD, PhD
—	—	—	—	BS, ME

Figure 4.15: Candidate screening list with match scores

Hireflow

Dashboard

Documents

Search

Ask

Candidates

Jobs

Activity Logs

Create Job

Create a new job posting with requirements

Basic Information

Job Title *

e.g. Senior Frontend Developer

Location

e.g. Remote, New York, NY

Job Description *

Describe the role, responsibilities, and expectations...

Skills & Requirements

Required Skills *

Add a skill and press Enter

Add

Preferred Skills

Add a preferred skill

Add

Education Level

any

Min Experience (years)

0

Max Experience (years)

5

Cancel

Create Job

Figure 4.16: Job creation with required skills, preferred skills, and an experience range

Department of Computer Science, SCET

39

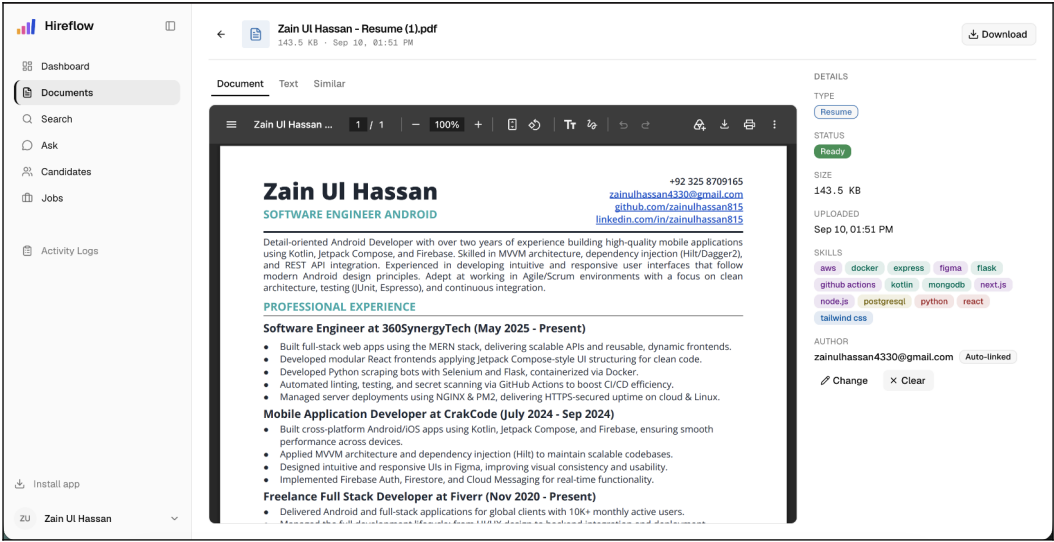


Figure 4.17: Document detail view with the in-app viewer and the extracted metadata

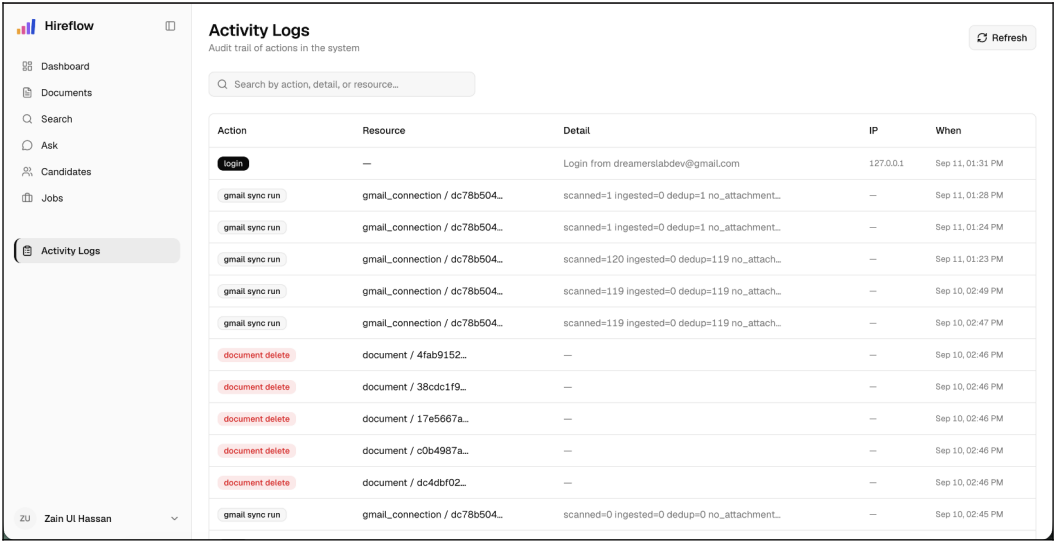


Figure 4.18: Activity log showing the audit trail of actions

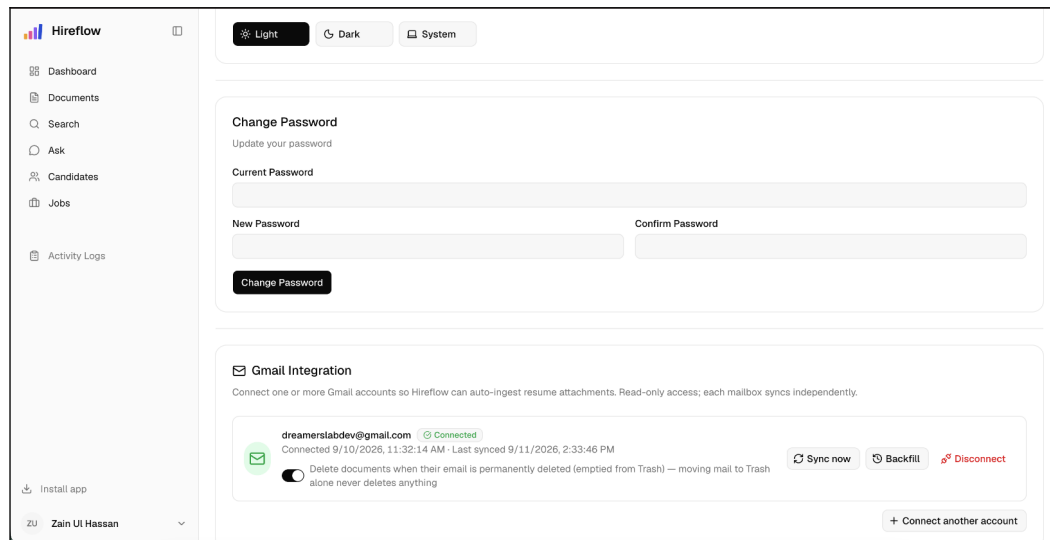


Figure 4.19: Settings, with the theme, password change, and the Gmail connection

CHAPTER 5: IMPLEMENTATION

This chapter records how the Hireflow platform was actually built from the architecture set out in the previous chapter. It walks through the local development environment and the way each service is started, the layered backend and the document-processing and RAG pipelines that form the core of the system, a handful of excerpts from the working codebase, and the database implementation across PostgreSQL and ChromaDB. Throughout, the focus stays on the system as it runs, not on how it was once imagined.

5.1 Development Environment Setup

Development targets a small cluster of containerised backing services alongside two application processes, the FastAPI API and a Celery worker, plus the Vite frontend dev server. The backing stores (PostgreSQL 15, Redis 7, ChromaDB, and MinIO) run under Docker Compose. The Python and Node processes, by contrast, run directly on the host, which keeps reloads fast during development.

Dependencies and the virtual environment on the backend are managed with uv; the frontend relies on npm with Vite. A typical bring-up, reproduced from the project guide, proceeds as follows:

```
# 1. Backing services (Docker)
docker compose up -d postgres redis chromadb minio

# 2. Backend API (from backend/)
uv sync
uv run uvicorn app.main:app --reload --port 8090

# 3. Celery worker (from backend/, separate terminal)
uv run celery -A app.worker.celery_app worker --loglevel=info

# 4. Frontend (from frontend/)
npm install
npm run dev                # Vite dev server on :5173
npm run generate-api        # regenerate the typed API client from
                             OpenAPI
```

First-time provisioning is wrapped in a `make setup` target, which installs dependencies, creates `.env`, starts the services, runs the Alembic migrations, and seeds the initial admin user; `make dev` then runs the API, worker, and frontend together. Configuration lives in one place, `core/config.py`, built on `pydantic-settings` so that a missing required variable fails fast. The most important of these is `JWT_SECRET_KEY`, which must be at least 32 characters. The development topology, with host ports exposed for convenience, is summarised below.

Table 5.1: Development Environment Services and Ports

Component	Runtime	Port	Purpose
Vite dev server	Host (npm)	5173	Frontend (React + TypeScript)
FastAPI API	Host (uvicorn)	8090	HTTP API under /api
Celery worker	Host (celery)	N/A	Background ingestion / re-index tasks
PostgreSQL 15	Docker	5432	Relational store + full-text search
Redis 7	Docker	6379	Token denylist, reset tokens, Celery broker
ChromaDB	Docker	8000	Vector store (chunk + candidate-summary embeddings)
MinIO	Docker	9000 / 9001	S3-compatible object storage (originals + viewables)

Production is far more closed. Under `docker-compose.prod.yml` only nginx port 80 is exposed. Nginx serves the built frontend and reverse-proxies `/api/` to the backend, while every data store sits on a private Docker network behind password and fail-fast guards.

5.2 Implementation Details

5.2.1 Layered Backend

The backend follows a strict layered architecture that keeps business rules independent of infrastructure. Each layer has an explicit import contract: `domain/` holds pure business rules and exceptions (stdlib only); `models/` holds the SQLAlchemy 2 ORM classes; `schemas/` holds Pydantic v2 request/response DTOs; `repositories/` encapsulate data access; `adapters/` provide Protocol definitions with swappable concrete implementations (password hashing, JWT, object storage, vector store, OCR/vision, LLM); `services/` coordinate the application

logic; and `api/routes/` expose thin HTTP handlers (target ≤ 5 lines each). One invariant matters above the rest: services never raise `HTTPException`. They raise `DomainError` subclasses, and a single error handler in `api/error_handlers.py` maps those to HTTP status codes, so that `NotFound` becomes a 404, `FileTooLarge` a 413, and `UnsupportedFileType` a 415. Adapters are wired to services in a composition root, `api/deps.py`, which exposes Annotated dependency aliases (`DocumentServiceDep`, `RagServiceDep`, `CurrentUser`, `RequireAdmin`, etc.) consumed by routes.

5.2.2 Request Validation and Error Handling

Every request is checked before it reaches business logic, and the checks sit in layers rather than in the handlers. Request bodies are Pydantic v2 schemas, so field rules such as minimum and maximum lengths, allowed enumerated values and required fields are enforced by the schema rather than by code in the route. Query parameters carry their own bounds at the route: a listing endpoint accepts a page size between one and one hundred and a non-negative offset, so a client cannot ask for an unbounded page. Uploads are guarded twice, once on size against a configured ceiling of ten megabytes and once on media type against an allowlist, and both guards run before the file is stored.

Access is checked on the same path. Protected routes depend on the current authenticated user, and administrator-only routes sit behind a separate guard. Beyond that, every service scopes its queries to the owner of the record. A request for something belonging to another user is answered as though the record does not exist, rather than as a refusal, because a refusal would confirm that the record is real. The same reasoning applies when linking records across owners.

Failures are handled in one place. Services never raise HTTP errors of their own; they raise domain exceptions, and a single handler translates those into status codes and into one response shape for the whole API, carrying a machine-readable code, a human-readable message, and optional details. A schema validation failure is reported as a 422 that lists the offending field and the reason, so a client can point at the input that was wrong. Anything unanticipated becomes a 500 with the detail logged on the server and kept out of the response. Table 5.2 lists the mapping.

Table 5.2: Domain Error to HTTP Status Mapping

Domain error	HTTP status	Raised when
Invalid credentials	401	Email or password is wrong. An unknown address returns the same shape, so accounts cannot be enumerated
Invalid token	401	Access or refresh token is

Domain error	HTTP status	Raised when
		missing, expired, or has been revoked
Account disabled	403	Credentials are valid but the account is deactivated
Forbidden	403	Authenticated, but the route requires a role the user does not hold
Not found	404	The record is missing, or belongs to another user
Document not indexed	404	Similar documents were requested for a document with no vector yet
Email already registered	409	Sign-up used an address that already has an account
Invalid status transition	409	An application was moved to a status it cannot reach from its current one
Resume already attached	409	The resume is already linked to a candidate
File too large	413	Upload exceeds the configured size limit
Unsupported file type	415	Media type is outside the upload allowlist
LLM rate limited	429	The language model provider refused for rate reasons; the response carries a retry delay
Gmail authorisation error	400	The Gmail OAuth exchange failed. The only caller is the browser-facing callback, which catches it and redirects to the settings page with a reason, so a client never

Domain error	HTTP status	Raised when
		receives the 400
Service unavailable	503	A dependency, or the language model provider, could not be reached
LLM timeout	504	The provider accepted the request but did not answer in time
Validation error	422	The request body or parameters failed schema validation

Every row of Table 5.2 was exercised against the running system rather than read off the source. Figures 5.1 to 5.16 show the request and the response for each one, captured with `curl` against the development stack. Two of them could not be produced by ordinary use: the rate-limit and timeout cases were created by pointing the language-model adapter at a stub that returns those conditions, and are marked as such below.



```
Invalid credentials · HTTP 401

$ curl -i -sS -X POST http://localhost:8090/api/auth/login -H \
  'Content-Type: application/json' -d \
  '{"email": "hr.demo@example.com", "password": "wrong-password"}'

HTTP/1.1 401 Unauthorized
content-type: application/json

{
  "error": {
    "code": "invalid_credentials",
    "message": "Invalid email or password."
  }
}

$ curl -i -sS -X POST http://localhost:8090/api/auth/login -H \
  'Content-Type: application/json' -d \
  '{"email": "nobody@example.com", "password": "wrong-password"}'

HTTP/1.1 401 Unauthorized
content-type: application/json

{
  "error": {
    "code": "invalid_credentials",
    "message": "Invalid email or password."
  }
}

Wrong password and unknown address return the identical body — accounts cannot be enumerated.
```

Figure 5.1: Invalid credentials (HTTP 401)

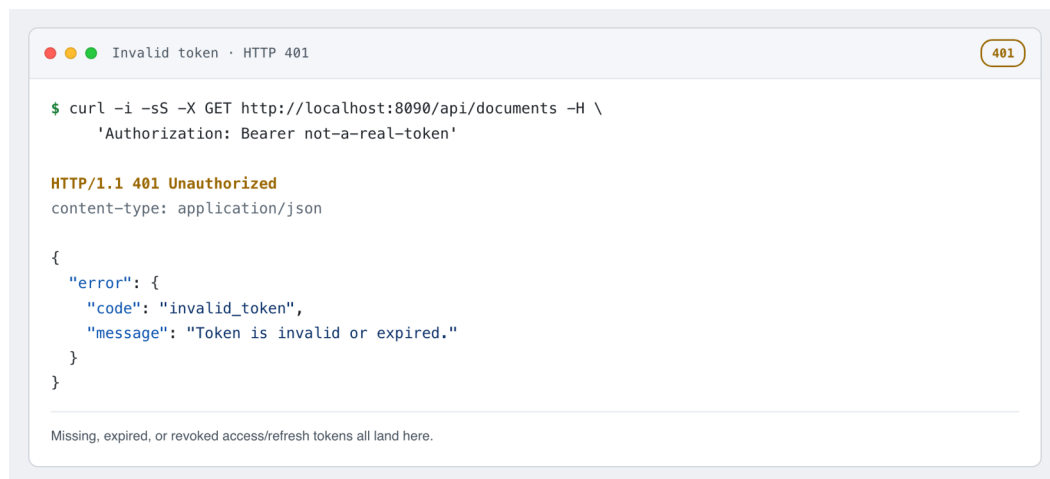


Figure 5.2: Invalid token (HTTP 401)



Figure 5.3: Account disabled (HTTP 403)



Figure 5.4: Forbidden (HTTP 403)



Figure 5.5: Not found (HTTP 404)



Figure 5.6: Document not indexed (HTTP 404)



Figure 5.7: Email already registered (HTTP 409)



```
Invalid status transition · HTTP 409

$ curl -i -sS -X PATCH \
  http://localhost:8090/api/jobs/4bcb554-0a6c-4048-857e-a395abb7b09b/status \
  -H 'Content-Type: application/json' -H 'Authorization: Bearer $TOKEN' -d \
  '{"status": "closed"}'

HTTP/1.1 409 Conflict
content-type: application/json

{
  "error": {
    "code": "invalid_status_transition",
    "message": "Cannot change status from 'draft' to 'closed'."
  }
}
```

draft -> closed is not a permitted lifecycle transition (draft -> open -> closed is).

Figure 5.8: Invalid status transition (HTTP 409)



```
Resume already attached · HTTP 409

$ curl -i -sS -X POST \
  http://localhost:8090/api/candidates/309eabab-4c9d-4f1a-ae2-dbf7faceb2100/attachments \
  -H 'Content-Type: application/json' -H 'Authorization: Bearer $TOKEN' -d \
  '{"attachments": [{"document_id": "145172d1-a9fa-4388-93df-da8ae4eeeb54a", "role": "resume"}, \
  {"document_id": "a60bbf01-9b16-4afc-bdfe-24dd6e3abcd2", "role": "resume"}]}'

HTTP/1.1 409 Conflict
content-type: application/json

{
  "error": {
    "code": "resume_already_attached",
    "message": "This candidate already has a resume. Detach it before attaching a new one."
  }
}
```

A candidate holds at most one resume attachment.

Figure 5.9: Resume already attached (HTTP 409)



```
File too large · HTTP 413

$ curl -i -sS -X POST http://localhost:8090/api/documents -H \
  'Authorization: Bearer $TOKEN' -F \
  'file=@oversized.pdf;type=application/pdf'

HTTP/1.1 413 Request Entity Too Large
content-type: application/json

{
  "error": {
    "code": "file_too_large",
    "message": "File exceeds 10 MB limit."
  }
}
```

11 MB upload against the 10 MB MAX_FILE_SIZE_MB limit.

Figure 5.10: File too large (HTTP 413)



Figure 5.11: Unsupported file type (HTTP 415)



Figure 5.12: LLM rate limited (HTTP 429)



Figure 5.13: Gmail authorisation error, served as a redirect to the settings page



A terminal window with a title bar that says "Service unavailable · HTTP 503". The window contains a curl command and its output. The curl command is: `$ curl -i -sS -X POST http://localhost:8090/api/rag/query -H \`
`'Content-Type: application/json' -H 'Authorization: Bearer $TOKEN' -d \`
`'{"question": "What skills does Ayesha Khan have?"}'`. The output shows an HTTP 1.1 503 Service Unavailable status, content-type: application/json, and a JSON error object: `{ "error": { "code": "llm_unavailable", "message": "Cannot reach the AI provider." } }`. At the bottom, a message reads: "LLM provider unreachable (authentication failure against the configured key)." A red "503" badge is in the top right corner.

```
$ curl -i -sS -X POST http://localhost:8090/api/rag/query -H \
'Content-Type: application/json' -H 'Authorization: Bearer $TOKEN' -d \
'{"question": "What skills does Ayesha Khan have?"}'

HTTP/1.1 503 Service Unavailable
content-type: application/json

{
  "error": {
    "code": "llm_unavailable",
    "message": "Cannot reach the AI provider."
  }
}

LLM provider unreachable (authentication failure against the configured key).
```

Figure 5.14: Service unavailable (HTTP 503)



A terminal window with a title bar that says "LLM timeout · HTTP 504". The window contains a curl command and its output. The curl command is: `$ curl -i -sS -X POST http://127.0.0.1:8091/api/rag/query -H \`
`'Content-Type: application/json' -H 'Authorization: Bearer $TOKEN' -d \`
`'{"question": "What skills does Ayesha Khan have?"}'`. The output shows an HTTP 1.1 504 Gateway Timeout status, content-type: application/json, and a JSON error object: `{ "error": { "code": "llm_timeout", "message": "AI provider timed out." } }`. At the bottom, a message reads: "Provider accepted the request and never answered inside the deadline." A red "504" badge is in the top right corner.

```
$ curl -i -sS -X POST http://127.0.0.1:8091/api/rag/query -H \
'Content-Type: application/json' -H 'Authorization: Bearer $TOKEN' -d \
'{"question": "What skills does Ayesha Khan have?"}'

HTTP/1.1 504 Gateway Timeout
content-type: application/json

{
  "error": {
    "code": "llm_timeout",
    "message": "AI provider timed out."
  }
}

Provider accepted the request and never answered inside the deadline.
```

Figure 5.15: LLM timeout (HTTP 504)



Figure 5.16: Validation error (HTTP 422)

5.2.3 Ingestion Pipeline

Ingestion is the path a file travels from the moment it is uploaded to the moment it becomes searchable and answerable. It runs asynchronously in the Celery worker so that the interface never blocks: the upload endpoint stores the raw file in MinIO, creates a document row with `status = PENDING`, returns HTTP 201 at once, and enqueues a Celery task that performs the remaining work in the background. The pipeline has seven stages, each consuming the output of the one before it, as summarised in Table 5.3 and depicted in Figure 4.4.

Table 5.3: Document Ingestion Pipeline Stages

#	Stage	Input	Processing	Output
1	Fetch	Document ID	The worker fetches the raw blob from MinIO object storage using the stored object key	Raw file bytes
2	Extract	Raw file bytes	The unstructured-based extractor performs layout-aware parsing: PyMuPDF for digital PDFs, and Tesseract/pytesseract OCR for scanned images and image-only pages	Typed elements (Title, NarrativeText, ListItem, Table) persisted to <code>document_elements</code>
3	Classify	Extracted text	A two-stage classifier runs rule-based keyword and structural heuristics first and falls back to the LLM only when rule confidence is	<code>document_type</code> (resume, certificate, report, ...) and extracted skills

#	Stage	Input	Processing	Output
			below 0.4	
4	Chunk	Typed elements	Heading-, table-, and list-aware chunking splits the text on semantic boundaries rather than fixed character counts, so a chunk does not straddle two sections	Chunks carrying section-heading and page-number metadata
5	Contextualise	Chunks	Each chunk is prefixed with its document title and section heading so that an isolated chunk still carries the context needed to be retrieved and cited correctly	Context-enriched chunk text
6	Embed	Enriched chunks	Chunks are batched through the sentence-transformers embedding provider (default bge-small-en-v1.5) to produce dense vectors	Embedding vector per chunk
7	Index	Vectors + chunk text	Vectors are upserted into a per-model ChromaDB collection while PostgreSQL's weighted search_tsv column is auto-populated for lexical search	Document searchable by both semantic and lexical paths

Once indexing succeeds the document is marked `status = READY`, and an on-ready hook fires: for a document classified as a resume, the hook automatically creates a candidate record from the extracted fields, so email-sourced and manually uploaded resumes both enter the candidate pool without further action.

Two design decisions make the pipeline maintainable in production. First, every stage is version-stamped with `extraction_version`, `chunking_version`, and `embedding_model_version`, so when a stage is improved a re-index can be limited to the documents whose stamps are stale, rather than reprocessing the whole corpus. Second, failure is handled by severity rather than uniformly: Celery tasks run with `acks_late = True` and up to three retries so a transient error such as a brief storage outage is retried rather than lost, while an indexing failure is treated as non-fatal, leaving the document at `READY` with a warning so that it remains viewable and lexically searchable even if its vectors are missing. Only an extraction failure, which leaves no text at all, marks the document as failed.

5.2.4 Search and RAG

Search merges four retrieval signals via Reciprocal Rank Fusion [29] ($k = 60$): vector similarity from ChromaDB, weighted lexical full-text search (`ts_rank_cd` over `search_tsv`, filename-A / skills-B / body-C), a SQL metadata path engaged only when structured filters are present, and a `pg_trgm` fuzzy fallback for typo tolerance. All paths respect per-user `owner_id` scoping (admin bypass) and `status = READY`.

Fusion is tuned rather than uniform: the lexical signal is weighted up so the filename and skills weighting carries through to the merged ranking. Reciprocal Rank Fusion is good at recall but leaves the order inside the top results unreliable, so a cross-encoder reranker [30] (BAAI/bge-reranker-base) reorders the leading candidates before they are returned. Retrieval widens to twenty candidates when the reranker is on, giving it room to reshuffle, and the model is kept off the main path by scoring only that shortlist rather than the whole corpus. The reranker can be switched off through configuration for comparison runs. The RAG service reuses this same retrieval path through a `ChunkRetriever` protocol, applies distance gates, runs an embedding-based intent classifier, composes a three-layer system prompt via `rag_prompts.py`, and streams a grounded answer from Anthropic Claude with typed citations, confidence, and intent on the wire.

5.2.5 Conversations and Follow-up Questions

Asking a question through `/rag/query` or `/rag/stream` is a single, self-contained exchange: the server keeps nothing, so a second question cannot refer back to the first. Because real users ask follow-ups, a conversation layer sits on top of that. A conversation is a chat thread private to its owner, stored in `conversations` with its individual turns in `chat_messages`, so a thread survives a page reload or a move to another machine.

Sending a question to `POST /conversations/{id}/messages` behaves like the streaming endpoint and emits the same events, with three differences. The server loads the earlier turns in the thread first and uses them to work out what a follow-up is actually referring to, so a question like “what about her education?” is resolved against the previous answer before retrieval runs. Both sides of the exchange are then written to the thread. Finally, a thread with no title yet is titled automatically once its first assistant reply lands, which is why a first turn that failed does not leave a thread permanently unnamed.

Deleting a thread is deliberately soft. Archiving hides it from the default listing but keeps it recoverable, because the thread is often the only record of how a hiring question was reasoned through. The original stateless endpoints are unchanged and still used by the evaluation harness.

5.2.6 Authentication and Storage

Authentication uses Argon2id password hashing with transparent rehash-on-login and timing-safe verification, JWT access (30 min) and refresh (7 days) tokens with rotation, and a Redis denylist for revoked refresh JTIs. Object storage sits behind a `BlobStorage` protocol implemented by MinIO, and signed URLs back the in-app document viewer.

5.2.7 Gmail Synchronisation

Hireflow connects to a user's mailbox through Gmail's OAuth 2.0 flow and keeps the resumes it finds there in step with the document library. One HR user can connect more than one mailbox: a connection is identified by the user together with the mail address rather than by the user alone, so a personal recruiting inbox and a shared careers address can both feed the same library. Each connection is synced separately and carries its own settings, including whether deletions are mirrored.

A sync run checks the mailbox for new messages with resume-style attachments. The list of accepted types is narrower here than for manual upload: PDF and Word only. Email bodies are full of signature logos and screenshots, and without that restriction every one of them would be pulled in, put through OCR, and left behind as a failed document. Each accepted attachment then goes through the same processing pipeline as a file uploaded by hand. The system keeps a record of every message it has already taken in, and a database constraint on the message identifier stops the same message being ingested twice. If a run does not use its whole budget on new mail, it spends what is left working backwards through older mail, one batch at a time, until the mailbox has been covered.

Keeping the two sides aligned also means noticing when mail disappears. Each connection remembers how far it has read through Gmail's record of changes, so a run can ask what has changed instead of scanning the whole mailbox again. If that position is too old for Gmail to answer from, the run starts tracking again from the present. When Gmail reports that a message Hireflow ingested has been moved to the Trash, the record for that message is stamped with the time it went away, and the stamp is cleared again if the user pulls the message back out. Moving a message to the Trash is reversible, so on its own it never removes anything from Hireflow.

Deletion is mirrored only when a message is permanently gone, either because the user emptied the Trash or because Gmail's own thirty-day purge removed it, and only on connections where the user has switched mirroring on. The setting is off by default, so an existing installation never starts deleting documents just because the feature was added. Where it does apply, the documents that message produced are removed properly: the stored file, its embeddings, and its link to any candidate all go. Every mirrored deletion is written to the activity log along with the message that caused it, so an audit can always answer why a document disappeared.

5.2.8 Frontend and API Client

The frontend is React 19 + TypeScript + Vite with Tailwind v4 and shadcn/ui, using TanStack Query for server state and React Context only for auth. The backend's OpenAPI spec is compiled into a fully typed SDK plus React Query hooks by @hey-api/openapi-ts (npm run generate-api), so the frontend consumes only generated types, with no hand-written request code or mock data. Components follow

composition-over-props conventions, and a client interceptor attaches the Bearer token and performs single-flight refresh on a 401 before replaying the request.

5.3 Code Snippets with Explanation

5.3.1 Timing-Safe Authentication

The authentication routine deliberately spends the password-verification cost even when no user exists, so that “unknown email” and “wrong password” cannot be told apart by response timing. That closes an account-enumeration side channel:

```
async def authenticate(self, *, email: str, password: str) ->
    User:
        user = await self._users.get_by_email(email)
        # Spend the verify cost even for a missing user so timing
        # is uniform.
        if user is None:
            self._hasher.verify(password, _DUMMY_HASH)
            raise InvalidCredentials("Invalid email or password.")
        if not self._hasher.verify(password,
            user.hashed_password):
            raise InvalidCredentials("Invalid email or password.")
        if not user.is_active:
            raise AccountDisabled("Account is disabled.")
        if self._hasher.needs_rehash(user.hashed_password): #
            # tighten Argon2 params
            user.hashed_password = self._hasher.hash(password)
            await self._users.save(user)
        return user
```

The service depends only on the PasswordHasher protocol and a repository, never on FastAPI, and it raises domain exceptions (`InvalidCredentials`, `AccountDisabled`) that the error handler later maps to 401/403. As a side benefit, it upgrades stored hashes opportunistically whenever Argon2 parameters tighten.

5.3.2 Weighted Candidate Scoring

Candidate-to-job matching blends four signals with fixed weights and rounds to a bounded score. The skill signal applies a configurable “required-skill floor”, so a missing required skill can gate the whole signal:

```
skill_score = self._skill_overlap(job, candidate)
exp_score = self._experience_fit(job, candidate)
vec_score = vector_scores.get(candidate.id, 0.0)
cred_score = self._credential_match(job, candidate)
```

```

return round(
    _WEIGHT_SKILLS * skill_score          # 0.40 - jaccard on
    required+preferred
    + _WEIGHT_EXPERIENCE * exp_score      # 0.20 - years-in-
    range fit
    + _WEIGHT_VECTOR * vec_score          # 0.30 - Chroma cosine
    vs job text
    + _WEIGHT_CREDENTIALS * cred_score, # 0.10 -
    cert/transcript coverage
    4,
)

```

The vector component comes from converting ChromaDB cosine distance into a similarity ($\max(0.0, 1.0 - \text{hit.distance})$) for the candidate's source resume. Each signal is also recorded in a `_breakdown()` dictionary that is persisted on the Application row, so the frontend hover-card can render the “why this score” explanation without any recomputation.

5.3.3 Self-Documenting Routes

Every route is thin and fully annotated for OpenAPI, which is the frontend developer's only contract. The upload handler reads the file, hands off to the document service, and returns a typed response:

```

@router.post(
    "", response_model=DocumentResponse, status_code=201,
    summary="Upload a document",
    responses={
        401: {"description": "Not authenticated"},
        413: {"description": "File exceeds the configured size
limit"},
        415: {"description": "File MIME type is not in the
allowed set"},
    },
)
async def upload_document(
    file: UploadFile, current_user: CurrentUser,
    documents: DocumentServiceDep, activity:
    ActivityServiceDep,
) -> DocumentResponse:
    data = await file.read()
    doc = await documents.upload(...)

```

The `summary`, `description`, and `responses` fields flow straight into the generated TypeScript SDK, so the frontend gets typed error semantics (413/415) for free.

5.4 Database Implementation

PostgreSQL 15 is the system of record, accessed through SQLAlchemy 2 in fully async mode over `asyncpg`, with ChromaDB kept separate as the vector store. The core ORM models (`User`, `Document`, `Job`, `Candidate`, `Application`, `ActivityLog`, `Conversation`, and `ChatMessage`, together with the supporting tables `document_elements`, `candidate_attachments`, `gmail_connections`, and `gmail_ingested_messages`) all inherit two mixins that standardise primary keys and timestamps:

```
class UUIDPrimaryKeyMixin:
    id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True), primary_key=True,
        default=uuid.uuid4,
    )

class TimestampMixin:
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(),
        nullable=False)
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(),
        onupdate=func.now(), nullable=False)
```

The `Document` model shows off the PostgreSQL-specific features in play: native enum columns (`document_status`, `document_type`) via `SAEnum` with a `values_callable` so the DB stores the string values; a `JSONB` metadata column for extracted fields (skills, experience, education); and a read-only, database-generated weighted `TSVECTOR` column (`search_tsv`) marked with `FetchdValue()` so the ORM never writes to it. That tsvector weights the filename (A), skills (B), and body text (C), which lets a filename match outrank a body-only mention without a single change to ranking code. Foreign keys carry explicit `ON DELETE CASCADE` policies from an owner to its documents, and `ON DELETE SET NULL` on the author linkage.

Alembic manages schema evolution, again in async mode. Downgrade scripts explicitly `DROP TYPE` for the PostgreSQL enums, and migrations such as the `match_breakdown` `JSONB` addition are versioned in the repository. Vector data lives outside PostgreSQL entirely. Chunk embeddings are upserted into per-embedding-model ChromaDB collections, where the collection name encodes the model, so swapping the embedding model creates a fresh collection and a startup integrity check warns on any mismatch; a separate whole-document / candidate-summary collection backs “similar documents” and candidate-lane RAG retrieval. Deleting a document cascades to its `document_elements` rows in PostgreSQL and removes the matching chunk vectors from ChromaDB, so the two stores stay consistent.

CHAPTER 6: TESTING & RESULTS

This chapter sets out how Hireflow was verified and what that verification showed. It describes the testing strategy across the automated and manual layers, presents concrete test cases and their outcomes taken from the project's QA test suite, discusses the performance of the retrieval and generation pipeline, and ends by weighing the results against the original objectives. The suite is organised by module (auth, documents, search, RAG, jobs, candidates, Gmail sync, settings, security, observability), with priorities running from P0 (critical) to P3 (nice-to-have).

6.1 Testing Strategy

Testing followed a layered strategy. At the base, unit tests run under `pytest` (`uv run pytest`) against services in isolation. The architecture's acceptance bar is blunt: any service must be instantiable in a test using fake adapters, with no Docker and no network, because services depend only on `Protocol` interfaces. Pure logic is therefore cheap to exercise directly, including the matching score blend, the skill-overlap floor, experience-fit decay, and the timing-safe authentication path.

Above that sit the integration and API tests, which drive full request/response cycles through FastAPI against the real backing services. They cover owner-scoping (cross-tenant access returning 404 to hide existence), the document ingestion pipeline reaching `READY`, RAG retrieval parity with the search pipeline, and the LLM error taxonomy mapping to the correct HTTP codes (503/429/504). Manual and UAT-style testing was carried out through the browser and `curl`, following the per-feature manual-test checklists recorded on the tracking issues. This layer covers the interactive surfaces that automated tests cannot easily assert: streaming chat rendering, drag-and-drop, the command palette, theme switching, and keyboard navigation.

Finally, static analysis and linting with `ruff` (`uv run ruff check --fix && uv run ruff format` on the backend, `npm run lint && npm run format` on the frontend) is a mandatory gate before any change is committed, under a zero-noqa policy. These layers together give confidence that both the business logic and the wired-up system behave as specified.

6.2 Test Cases and Results

Table 6.1 presents a representative slice of the full suite, spanning authentication, document processing, hybrid search, RAG question-answering, and candidate matching. Each case names the use case it exercises and gives an example input, the expected result, and the actual observed result; all reflect the specified and verified behaviour of the implemented system.

Table 6.1: Test Cases and Results

Use Case	Test Case	Input	Expected Result	Actual Result	Status
UC-01	TC-AUTH-006: Login with valid credentials issues tokens	POST /api/auth/login with email hr@test.hireflow.io and password hr-test-password	HTTP 200; access + refresh token in body; token_type=bearer	Both tokens returned; token_type=bearer	Pass
UC-01	TC-AUTH-008: Unknown email returns invalid_credentials (no user enumeration)	POST /api/auth/login with email nobody@test.hireflow.io and any password	HTTP 401 with identical shape/message to the wrong-password case	401 with identical envelope; response timing uniform	Pass
UC-01	TC-AUTH-014: Refresh rotates token; old refresh becomes unusable	POST /api/auth/refresh with a valid refresh token, then the same request again reusing that same token	Step 1 returns HTTP 200 with a new pair; step 2 returns HTTP 401 invalid_token	Old token revoked in Redis; reuse rejected with 401	Pass
UC-03	TC-DOCS-001: Uploaded PDF appears in list as ready	POST /api/documents with alice_resume.pdf (two-page PDF, 180 KB), then poll the document list	The row appears with status=ready within 60 seconds	Pipeline settled to ready; detail view opens	Pass
UC-03	TC-DOCS-004: Upload over the size limit returns 413	POST /api/documents with an 11 MB scanned PDF, against the 10 MB limit	HTTP 413 with error code file_too_large	413 file_too_large returned	Pass
UC-03	TC-DOCS-005: Unsupported MIME type returns 415	POST /api/documents with candidates.csv, media type text/csv	HTTP 415, unsupported file type	415 returned; allowlist enforced	Pass
UC-09	TC-DOCS-046: Candidate auto-created from a resume	POST /api/documents with alice_resume.pdf, wait for status ready, then GET /api/candidates	A candidate row linked via source_document_id	Candidate auto-created by the on-ready hook	Pass
UC-04	TC-SEARCH-005: An HR user sees only their own documents	Signed in as hr@test.hireflow.io, search for admin_python, a term that appears only in the administrator account documents	No results	Owner scope enforced; empty result set	Pass
UC-04	TC-SEARCH-015: Trigram fallback fires when full-text search yields nothing	Search for pyhton, a transposition of python, where no filename contains either spelling	The document returns via the word_similarity fuzzy fallback	Fuzzy path recovered the python document	Pass
UC-04	TC-RAG-001: Grounded answer returned with	POST /api/rag/query with the question: Which candidates have	HTTP 200; answer cites at least one source	Answer grounded with	Pass

Use Case	Test Case	Input	Expected Result	Actual Result	Status
	citations	Kubernetes experience?	[filename]; confidence non-null	typed citations	
UC-04	TC-RAG-002: Out-of-scope question returns the refusal sentinel	POST /api/rag/query with the question: Explain quantum physics	HTTP 200; exact sentinel; confidence null; no fabricated citations	Sentinel “Not in the provided documents.” returned	Pass
UC-12	TC-JOBS-010: Match scores candidates with a breakdown	Run match for a job requiring Python, Django and PostgreSQL against eight indexed resumes	Each result has a score plus a breakdown (skill, experience, vector)	Scores and persisted match_breakdown returned	Pass

The traceability matrix below links a sample of use cases and their functional requirements to the test cases that exercise them.

Table 6.2: Test Case Traceability Matrix

Use Case ID	Use Case Name	Functional Requirement ID	Test Case ID(s)	Test Scenario	Status
UC-01	Login	FR01, FR03	TC-AUTH-006, TC-AUTH-008, TC-AUTH-014	Login, no-enumeration, token rotation	Pass
UC-03	Upload Document	FR04, FR05, FR06	TC-DOCS-001, TC-DOCS-004, TC-DOCS-005	Ingestion to ready , size + MIME guards	Pass
UC-04	Search Documents / Ask	FR07, FR08, FR09	TC-SEARCH-005, TC-SEARCH-015, TC-RAG-001, TC-RAG-002	Hybrid search, typo tolerance, grounded RAG, sentinel	Pass
UC-12	Read Resumes / Screen	FR13, FR14, FR15	TC-JOBS-010	Weighted scoring + bounded score	Pass
UC-09	Sync Resumes	FR17, FR18	TC-DOCS-046	On-ready candidate creation from resume	Pass

6.3 Performance Evaluation

Performance was measured on the development stack rather than a production server. The test machine was a single laptop running WSL2 with a 13th-generation Intel Core i5-13450HX (16 logical cores) and about 7.6 GiB of RAM allocated to the WSL2 virtual machine. The sentence-transformer embedder ran on an NVIDIA RTX 5050 laptop GPU, the RAG model was the hosted Claude Haiku 4.5, and the corpus held 18 ready documents indexed as 49 chunks. All timings use a monotonic clock, with the

first call on each path discarded as warm-up. Table 6.3 summarises the results, which are discussed below.

Table 6.3: Measured Performance (Development Stack)

Path	Metric	Result
Hybrid search (server-side, excludes LLM)	median / 95th-percentile latency	72 ms / 167 ms (N = 31)
Ingestion, text PDF	upload to READY	44.1 s avg (5.1 to 103.1 s, page-driven)
Ingestion, scanned image (OCR path)	upload to READY	4.1 s (single page)
Sustained ingestion (batch, worker concurrency 1)	throughput	1.5 documents / minute
Chunk embedding (GPU, batch 128)	per chunk	0.57 ms
RAG streaming answer	time-to-first-token	947 ms avg (749 to 1573 ms)
RAG streaming answer	full completion	1.5 s avg (1.1 to 1.9 s)

6.3.1 Retrieval Latency

Hybrid search combines a ChromaDB vector query, a PostgreSQL weighted full-text query, an optional SQL-metadata query, and a fuzzy fallback, all fused via RRF. The lexical and metadata paths run against indexed PostgreSQL columns (a GIN index on `search_tsv`, JSONB containment for skills), and the vector query is bounded to a small `n_results`, so end-to-end search stays interactive. Over 31 warm queries, the server-side search latency (excluding LLM generation) had a median of 72 ms and a 95th percentile of 167 ms, with a mean of 86 ms and a range of 28 to 261 ms. The one-off cold call that loads the query-embedding model took about 8 seconds and was excluded. These figures reflect the current small index; retrieval latency will grow as the corpus scales.

6.3.2 Ingestion Throughput

Ingestion runs asynchronously in the Celery worker and is dominated by document extraction rather than API time. Upload returns immediately with `status=pending`, and the time to reach `READY` is driven mainly by page count, because the hi-res layout model in the extraction stage runs once per page. Across five text-based PDFs the mean was 44.1 seconds, but the spread was wide: 5.1 seconds for a single-page resume up to 103.1 seconds for a long, complex multi-page document. Three

synthesised single-page scanned images that exercised the Tesseract OCR path processed in about 4.1 seconds each, so a simple scanned page is in fact faster than a complex multi-page PDF, and OCR is not the bottleneck. With the Celery worker at concurrency 1, a batch of five documents uploaded together was processed serially at a sustained rate of about 1.5 documents per minute; raising worker concurrency would improve this.

6.3.3 Embedding and LLM Response Times

Embeddings use `bge-small-en-v1.5`, a small sentence-transformers model. On the RTX 5050 GPU with a batch size of 128, embedding cost a median of 0.57 ms per chunk, so this stage is inexpensive and, in this environment, GPU-bound rather than CPU-bound. The dominant RAG latency is the Claude generation, streamed token-by-token over SSE so that perceived latency stays low. Across 11 warm questions the average time-to-first-token was 947 ms (749 to 1573 ms) and the average full-answer completion was 1.5 seconds (1.1 to 1.9 s); the server-side end-to-end figure, which includes retrieval and generation, reached up to about 2.75 seconds. Because the LLM is hosted, these times also carry network and provider variance.

6.4 Results and Discussion

Measured against the project's objectives, Hireflow delivers the four capabilities set out in the brief. HR personnel can manage unstructured documents: the ingestion pipeline reliably parses PDFs, DOCX, and images (with OCR), classifies them, and surfaces them through an in-app multi-format viewer, reaching READY within the UX budget and recovering on its own from non-fatal indexing failures. They can search and ask natural-language questions through a hybrid retrieval engine that fuses vector, lexical, metadata, and fuzzy signals, backed by a RAG service that returns grounded, cited answers and, when a question falls outside the corpus, refuses it with an exact sentinel rather than hallucinating. They can screen and rank resumes against jobs using a transparent four-signal weighted score with a persisted, explainable breakdown. And candidates are synced automatically, both from uploaded resumes (on-ready hook) and from Gmail via encrypted OAuth connections with deduplication.

Feature completeness aside, the testing exercise validated the system's shared guarantees: strict per-user owner scoping (cross-tenant access returns 404 to hide existence, with a documented admin bypass), a timing-safe, non-enumerable auth surface with token rotation, a consistent error envelope, and a fully typed frontend-to-backend contract generated from OpenAPI. The layered architecture, with services depending only on protocols, earned its keep by leaving the core logic unit-testable without any infrastructure.

Testing also brought the gaps into the open rather than papering over them. Login has no rate-limiting or lockout, a password change does not revoke existing refresh tokens, and a few upload edge cases (a zero-byte file, for one) are not yet guarded.

These are recorded as known behaviours and hardening candidates, not as defects that block the core workflow. On balance, the results point to a working, coherent system that meets its functional objectives, with a clear and prioritised backlog of security-hardening improvements left for future work.

CHAPTER 7: CONCLUSION & FUTURE WORK

This chapter looks back on the project as delivered. It weighs the outcome against the objectives set out in Chapter 1, is honest about where the current system falls short, and sketches the enhancements that would follow with more time in hand.

7.1 Conclusion

Hireflow began with a simple frustration: managing unstructured HR documents and screening resumes by hand is slow and error-prone. The system set out to ease that burden by bringing document understanding, semantic retrieval, and large-language-model generation together in one coherent, self-hostable platform. It meets the core objectives defined in Chapter 1. Heterogeneous documents, whether PDF, Word, or scanned image, are ingested through an OCR and text-extraction pipeline. Their content is then chunked, embedded, and indexed in ChromaDB, so that a natural-language query returns relevance-ranked results. On top of this sits a Retrieval-Augmented Generation service that answers plain-language questions with source-attributed citations, and declines out-of-scope questions instead of inventing an answer. Candidate resumes are screened and ranked against job criteria through a transparent, weighted scoring model, while intake itself is automated by way of Gmail synchronisation, shortlisting, and Excel export.

The engineering matters as much as the features. The system rests on a disciplined, layered architecture: business logic is kept apart from infrastructure behind Protocol-based adapters. That separation makes each provider (LLM, embeddings, OCR, storage) swappable, and it leaves the core logic unit-testable without any external service standing by. What the project delivers, then, is twofold. It is a working AI-assisted HR tool, and it is a sound approach to weaving OCR, dense-vector retrieval, and LLM generation into a production-shaped whole.

7.2 Limitations

Despite meeting its functional goals, the current version has several limitations:

- **Language coverage.** Document processing and retrieval are tuned for English, and multilingual documents are not explicitly supported.
- **OCR accuracy on poor inputs.** Text extraction is reliable on standard printed material, but it degrades on low-quality scans and handwritten content, an inherent limitation of the OCR stage.
- **Security hardening gaps.** Login lacks rate-limiting and account lockout, a password change does not revoke existing refresh tokens, and some upload edge cases are not yet guarded; these are documented rather than resolved.
- **Dependence on an external LLM.** RAG answer quality and latency depend on the configured LLM provider, and hosted-model usage introduces variable cost and network dependency, although an on-premise fallback exists.

- **Evaluation depth.** Retrieval quality is measured against a small labelled fixture, which is large enough to tune the pipeline but too small to prove that the results generalise. A larger, more varied corpus would be needed before the retrieval scores could be reported with confidence.
- **Scope boundaries.** Third-party ATS integration, video/audio processing, and native mobile applications are out of scope for this version.

7.3 Future Enhancements

With additional time, the following enhancements would strengthen the system:

- **Security hardening:** add login rate-limiting and lockout, refresh-token revocation on password change, and zero-byte/edge-case upload guards.
- **Multi-language support:** extend OCR and embeddings to additional languages, with language detection during ingestion.
- **Quantitative evaluation:** capture controlled benchmarks for retrieval latency, ingestion throughput, and RAG relevance, and add automated regression tests for answer quality.
- **Third-party ATS integration:** integrate with external Applicant Tracking Systems to import and export candidate pipelines.
- **Analytics dashboards:** expand recruiter-facing analytics (funnel metrics, time-to-shortlist, skill-gap analysis) using the existing charting layer.
- **Real-time notifications:** notify users when Gmail sync completes, when documents finish processing, or when strong candidates are matched.
- **Native mobile application:** provide a mobile client for on-the-go review and shortlisting.
- **Domain-adapted models:** fine-tune or select embeddings specialised for resumes and HR documents to improve retrieval and ranking quality.

REFERENCES

References are numbered in the order in which they are first cited in the text, and are referred to by a bracketed number such as [1]. Entries follow IEEE reference style.

1. Google Cloud. "Document AI overview." Google Cloud. <https://cloud.google.com/document-ai> (accessed Sep. 11, 2026).
2. Amazon Web Services. "Amazon Textract: Extract text and data from documents." AWS. <https://aws.amazon.com/textract> (accessed Sep. 11, 2026).
3. Microsoft. "Azure AI Document Intelligence documentation." Microsoft Azure. <https://azure.microsoft.com/en-us/products/ai-services/ai-document-intelligence> (accessed Sep. 11, 2026).
4. R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, vol. 2, 2007, pp. 629-633.
5. Artifex Software. "PyMuPDF documentation." Artifex. <https://pymupdf.readthedocs.io> (accessed Sep. 11, 2026).
6. J. Johnson, M. Douze, and H. Jegou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, 2021.
7. Hugging Face. "Document AI: Understanding documents with transformers." Hugging Face. <https://huggingface.co/blog/document-ai> (accessed Sep. 11, 2026).
8. A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998-6008.
9. P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459-9474.
10. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982-3992.
11. Anthropic. "Claude documentation." Anthropic. <https://docs.anthropic.com> (accessed Sep. 11, 2026).
12. D. Hardt, "The OAuth 2.0 authorization framework," IETF RFC 6749, Oct. 2012.
13. Greenhouse Software. "Greenhouse applicant tracking system." Greenhouse. <https://www.greenhouse.io> (accessed Sep. 11, 2026).
14. Workday. "Workday Recruiting." Workday. <https://www.workday.com> (accessed Sep. 11, 2026).
15. Eightfold AI. "Eightfold AI talent intelligence platform." Eightfold AI. <https://eightfold.ai> (accessed Sep. 11, 2026).
16. LangChain. "LangChain documentation." LangChain. <https://python.langchain.com> (accessed Sep. 11, 2026).

17. *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998.
18. M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.
19. A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 memory-hard function for password hashing and proof-of-work applications,” IETF RFC 9106, Sep. 2021.
20. S. Ramirez. “FastAPI documentation.” FastAPI. <https://fastapi.tiangolo.com> (accessed Sep. 11, 2026).
21. PostgreSQL Global Development Group. “PostgreSQL 15 documentation.” PostgreSQL. <https://www.postgresql.org/docs/15/> (accessed Sep. 11, 2026).
22. Chroma. “Chroma documentation.” Chroma. <https://docs.trychroma.com> (accessed Sep. 11, 2026).
23. Celery Project. “Celery documentation.” Celery. <https://docs.celeryq.dev> (accessed Sep. 11, 2026).
24. Redis. “Redis documentation.” Redis. <https://redis.io/docs/latest/> (accessed Sep. 11, 2026).
25. MinIO. “MinIO object storage documentation.” MinIO. <https://min.io/docs/minio/linux/index.html> (accessed Sep. 11, 2026).
26. S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, “C-Pack: Packed resources for general Chinese embeddings,” arXiv:2309.07597, 2023.
27. Meta. “React documentation.” React. <https://react.dev> (accessed Sep. 11, 2026).
28. Docker. “Docker documentation.” Docker. <https://docs.docker.com> (accessed Sep. 11, 2026).
29. G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2009, pp. 758-759.
30. R. Nogueira and K. Cho, “Passage re-ranking with BERT,” arXiv:1901.04085, 2019.

APPENDICES

Appendix A: User Manual

A complete step-by-step user manual is provided as a separate document accompanying this report. It covers sign-up and login, uploading documents, searching and asking questions, creating jobs, screening candidates, connecting Gmail, and exporting shortlists. The main user interfaces are also illustrated in Chapter 4 (Figures 4.11-4.19) and Chapter 6.

Appendix B: Sample Code

Representative source listings are provided in Chapter 5 (Sections 5.3 and 5.4), including the timing-safe authentication routine, the weighted candidate-scoring blend, a self-documenting upload route, and the shared ORM mixins. The complete source code, including the backend, frontend, and deployment configuration, is available in the project's public GitHub repository:

<https://github.com/zainulhassan815/hireflow>

Appendix C: Gantt Chart (FYP-I and FYP-II)

The project schedule across both FYP phases is summarised below. Filled cells (■) indicate the months in which each activity was active.

Activities / Tasks	M1	M2	M3	M4	M5	M6	M7	M8
Project Topic Selection	■							
Problem Identification	■	■						
Literature Review		■	■					
Requirements Gathering & Feasibility Study		■	■					
Software Requirements Specification (SRS)			■	■				
System Design (UML, Database,				■	■			

Activities / Tasks	M1	M2	M3	M4	M5	M6	M7	M8
UI)								
Prototype Development				■	■			
Database Development					■	■		
Frontend Development					■	■	■	
Backend Development					■	■	■	
System Integration & Testing						■	■	■
Unit Testing						■	■	
User Acceptance Testing (UAT)							■	■
Bug Fixing & Optimisation							■	■
Documentation							■	■
Final Report Writing								■
Final Presentation & Submission								■